MiniMax Sparse Attention

Xunhao Lai[1,2], **Weiqi Xu**[1], **Yufeng Yang**[1], **Qiaorui Chen**[3], **Yang Xu**[1,4], **Lunbin Zeng**[1,5], **Xiaolong Li**[1,4], **Haohai Sun**[1], **Haichao Zhu**[1], **Vito Zhang**[1,2] **and Pengyu Zhao**[1]

[1]MiniMax, [2]Peking University, [3]NVIDIA, [4]Zhejiang University, [5]Huazhong University of Science and Technology

Ultra-long-context capability is becoming indispensable for frontier LLMs: agentic workflows, repository-scale code reasoning, and persistent memory all require the model to jointly attend over hundreds of thousands to millions of tokens—yet the quadratic cost of softmax attention makes this untenable at deployment scale. We introduce MiniMax Sparse Attention (MSA), a blockwise sparse attention built upon Grouped Query Attention (GQA). A lightweight Index Branch scores key–value blocks and independently selects a Top-k subset for each GQA group, enabling group-specific sparse retrieval while maintaining efficient block-level execution; the Main Branch then performs exact block-sparse attention over only the selected blocks. Designed around a principle of simplicity and scalability, MSA is deliberately streamlined, making it straightforward to deploy efficiently across a broad range of GPUs. To translate sparsity into practical speedups, we co-design MSA with a GPU execution path that uses exp-free Top-k selection and KV-outer sparse attention to improve tensor-core utilization under block-granular access. On a 109B-parameter model with native multimodal training, MSA performs on par with GQA while reducing per-token attention compute by 28.4× at 1M context. Paired with our co-designed kernel, MSA achieves 14.2× prefill and 7.6× decoding wall-clock speedups on H800. Our inference kernel is available at: https://github.com/MiniMax-AI/MSA. A production-grade natively multimodal model powered by MSA has been publicly released at: https://huggingface.co/MiniMaxAI/MiniMax-M3.

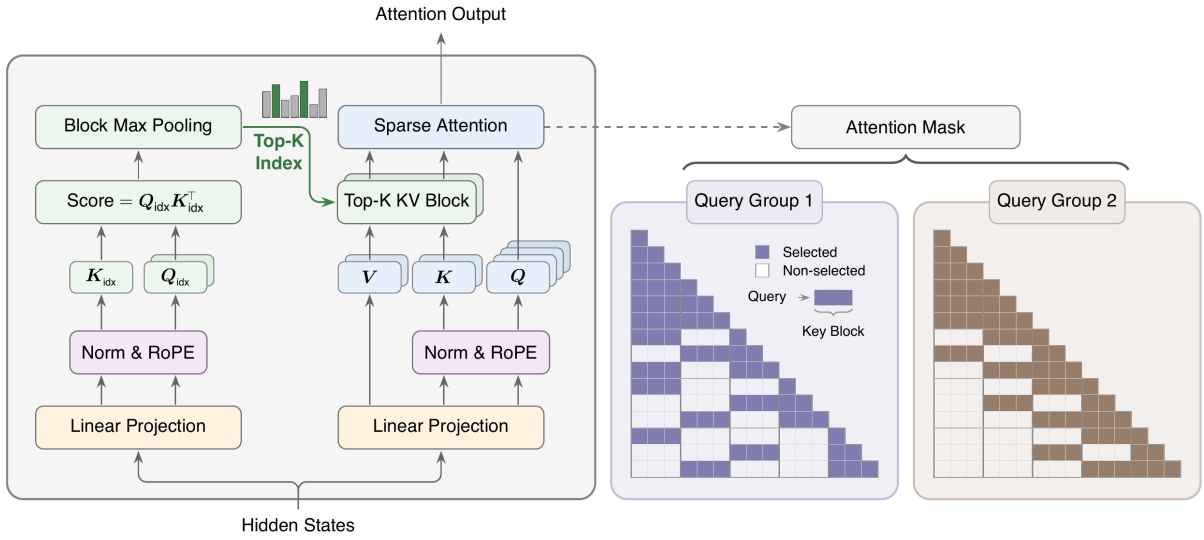


Figure 1 | Overview of MSA. The Index Branch (left) scores the full causal context with a single lightweight head and selects, for each query and GQA group, a set $\mathcal{I}$ of k key blocks; the local block is always included regardless of its score. The Main Branch (right) attends only to the selected blocks and produces the layer output. During training, a KL loss aligns the index distribution with the group-averaged Main Branch distribution on the selected blocks, and the Index Branch gradient is detached from the Main Branch.

© 2026 MiniMax. All rights reserved

1. Introduction

Large language models (LLMs) are rapidly shifting from short, single-turn interactions to long-horizon agentic workflows that span hundreds of interleaved reasoning and action steps—writing and deploying production code, navigating the open web, orchestrating diverse tools, and producing structured documents (Anthropic, 2025; DeepSeek-AI, 2026; Google DeepMind, 2025; Moonshot AI, 2026; OpenAI, 2025; Zhipu AI, 2026). However, the ultra-long contexts these tasks demand impose severe compute and memory bottlenecks on both training and inference, with quadratic-cost softmax attention being the primary culprit, further amplified by the latency and throughput constraints of production-scale deployment.

Context length is a critical scaling dimension for LLMs, where trading off model quality against efficiency remains a formidable challenge. The community is actively pushing the Pareto frontier on this front. Hybrid architectures (MiniMax, 2025b; Qwen, 2026) replace a subset of softmax attention layers with efficient alternatives such as linear attention (Gu and Dao, 2023; Team et al., 2025a; Yang et al., 2025) or sliding window attention (MiMo et al., 2026; OpenAI et al., 2025). Alternatively, another line of work attempts to sparsify softmax attention (DeepSeek-AI, 2026; DeepSeek-AI et al., 2025; Lu et al., 2025; Team et al., 2025b) itself to break the computational bottleneck.

We introduce MiniMax Sparse Attention (MSA), designed following Occam's razor: after extensive ablation, we retain only the essential components. MSA follows the sparse softmax attention paradigm to maximally reuse existing software and hardware infrastructure. We adopt blockwise token selection with a smaller top-k, enabling efficient execution across a wider range of GPU architectures while relaxing the head-dimension constraints imposed by prior designs. Concretely, an ultra-lightweight Index Branch selects, for each attention group, the top-k blocks via max-pooling scoring, while always retaining the most recent block to ensure training stability.

Turning MSA's theoretical sparsity into practical end-to-end speedups requires co-designing the algorithm with its GPU execution path. To this end, we design an exp-free TopK kernel specialized for the small-k regime, leveraging the blockwise indexer to bypass unnecessary softmax computation before selection. For the main attention branch, we organize sparse attention in a KV-outer order: selected KV blocks gather their associated queries and concatenate them to fill tensor-core MMAs, using pre-scheduled chunking with a two-phase combine to handle highly skewed block popularity without atomic updates. For training, we further fuse the auxiliary LSE computation required by the sparse KL loss into the forward pass and employ persistent load balancing in the backward pass.

To validate that MSA preserves both textual and multimodal capabilities, we compare it against Grouped Query Attention on a 109B-parameter Mixture of Experts (MoE) model trained from scratch with a 3T-token budget. MSA matches GQA on downstream benchmarks while delivering 14.2× prefill and 7.6× decoding speedups at 1M context length.

Main contributions.

- We propose MSA, a minimal, scalable, and accelerated blockwise sparse attention mechanism that supports both training from scratch and near-lossless conversion from pretrained GQA checkpoints.
- We co-design efficient training and inference kernels that turn MSA's theoretical compute savings into real wall-clock speedups at scale.
- We perform extensive ablations scaling up to a 109B-parameter MoE model with native multimodal training, dissecting MSA's behavior across scales and modalities.

2. Preliminary

2.1. Causal Attention and GQA

We write N for the sequence length, d_{model} for the hidden dimension, and d_h for the head dimension. For each query position t and head h, causal Softmax Attention computes

$$\boldsymbol{o}_t^{(h)} = \sum_{i \le t} \alpha_{t,i}^{(h)} \boldsymbol{v}_i^{(h)}, \qquad \alpha_{t,i}^{(h)} = \frac{\exp\left(\langle \boldsymbol{q}_t^{(h)}, \boldsymbol{k}_i^{(h)} \rangle / \sqrt{d_h}\right)}{\sum_{j \le t} \exp\left(\langle \boldsymbol{q}_t^{(h)}, \boldsymbol{k}_j^{(h)} \rangle / \sqrt{d_h}\right)}. \tag{1}$$

The cost of eq. (1) is $\Theta(2H_q N^2 d_h)$ FLOPs, which grows quadratically with the sequence length N. Grouped-Query Attention (Ainslie et al., 2023) uses H_q query heads and reduces the number of key-value heads to H_{kv}, tying $G = H_q / H_{kv}$ adjacent query heads to a single shared key-value head. Thus, each key-value head defines one GQA group.

2.2. Sparse Attention as a Two-Stage Process

A sparse attention layer factors causal attention into an indexer that selects which keys to attend to and a sparse attention computation over the selected keys. For each query position i,

$$\mathcal{I}_i = \text{Index}_\phi\left(\boldsymbol{q}_i, \boldsymbol{K}_{\le i}\right), \qquad \boldsymbol{o}_i = \text{Attn}\left(\boldsymbol{q}_i, \boldsymbol{K}[\mathcal{I}_i], \boldsymbol{V}[\mathcal{I}_i]\right), \tag{2}$$

where Index_ϕ is parameterized by ϕ (empty for fixed-rule indexers; learned for trainable ones), $\mathcal{I}_i \subseteq \{1, \dots, i\}$ denotes the selected index set, and Attn denotes standard scaled dot-product softmax attention restricted to this index set. We call the first stage the *Index Branch* and the second the *Main Branch*. In multi-head attention, each query, specified by a position i and a query head h, can select a different key/value index set, written as $\mathcal{I}_i^{(h)}$; eq. (2) omits the head index only for notational simplicity.

2.3. GQA-Based Block Sparse Attention

Per-head token-level selection offers the finest granularity, but such fine-grained computation is difficult to map efficiently to GPU matrix operations. For efficiency, sparse attention built on GQA can share the index result within each GQA group. Let $\mathcal{H}_r$ denote the G query heads served by the r-th key-value head. The group-shared index set can be written as

$$\mathcal{I}_i^{(r)} = \mathcal{I}_i^{(h)} = \mathcal{I}_i^{(h')}, \qquad h, h' \in \mathcal{H}_r. \tag{3}$$

Selecting key/value blocks rather than individual tokens reduces routing overhead and makes sparse attention more regular. For block size B_k, define

$$\mathcal{B}_b = \{(b-1)B_k + 1, \dots, \min(bB_k, N)\}, \qquad b = 1, \dots, B, \quad B = \lceil N / B_k \rceil. \tag{4}$$

For query position i and GQA group r, the set $\mathcal{I}_i^{(r)} \subseteq \{1, \dots, B\}$ denotes the selected block index set. The sparse attention output for any query head in group r is then computed over the causally visible tokens in the selected blocks, using the key-value head of the same group. MSA follows this GQA-based block sparse formulation, with the concrete indexer architecture and training objective described in the next section.

3. MSA

We introduce MiniMax Sparse Attention (MSA), a GQA-based sparse attention mechanism with two branches, as illustrated in Figure 1. For each query token, a lightweight *Index Branch* selects a small set of key blocks from the causal context, and the *Main Branch* computes softmax attention over the tokens in those blocks. The Index Branch adds only two projection matrices to standard GQA, operates at block granularity, and makes selections independently for each GQA group. We describe the architecture in Section 3.1 and the training procedure in Section 3.2.

3.1. Architecture

MSA instantiates the two-stage sparse-attention formulation in Section 2.2 at GQA-group and block granularity (Figure 1). For each query token, the Index Branch selects k key blocks of size B_k for each GQA group, and the Main Branch attends only to tokens in the selected blocks, whose budget is at most kB_k. Let $\boldsymbol{X} \in \mathbb{R}^{N \times d_{\text{model}}}$ be the input hidden states. Following Section 2.1, we write H_q and H_{kv} for the number of query heads and key-value heads, respectively, so each key-value head serves $G = H_q / H_{kv}$ query heads.

Index Branch. The Index Branch introduces one index query head for each GQA group and a single index key head shared across groups:

$$\boldsymbol{Q}^{\text{idx}} = \boldsymbol{X}\boldsymbol{W}_q^{\text{idx}} \in \mathbb{R}^{N \times H_{kv} \times d_{\text{idx}}}, \qquad \boldsymbol{K}^{\text{idx}} = \boldsymbol{X}\boldsymbol{W}_k^{\text{idx}} \in \mathbb{R}^{N \times 1 \times d_{\text{idx}}}. \tag{5}$$

For query token i and group r, the Index Branch first scores visible key tokens, then aggregates these scores to the block level. Using the block partition $\mathcal{B}_1, \ldots, \mathcal{B}_B$ defined in Section 2.3,

$$\boldsymbol{S}_{i,j}^{\text{idx},(\mathrm{r})} = \frac{\left(\boldsymbol{Q}^{\text{idx}}\right)_i^{(r)} \left(\boldsymbol{K}^{\text{idx}}\right)_j^{\top}}{\sqrt{d_{\text{idx}}}}, \qquad M_{i,b}^{\text{idx},(\mathrm{r})} = \max_{\substack{j \in \mathcal{B}_b \\ j \le i}} \boldsymbol{S}_{i,j}^{\text{idx},(\mathrm{r})}. \tag{6}$$

Here r indexes the GQA group, $j \le i$ enforces causality, and blocks with no visible token are assigned score $-\infty$. The Index Branch then selects the top-k block indices:

$$\mathcal{I}_i^{(r)} = \mathrm{TopK}_{b \in \{1,\ldots,B\}}\left(M_{i,\cdot}^{\text{idx},(\mathrm{r})}, k\right). \tag{7}$$

Here $\mathrm{TopK}(\cdot, k)$ returns the indices of the k largest blocks under $M_{i,\cdot}^{\text{idx},(\mathrm{r})}$. We always include the local block containing position i, and $\mathcal{I}_i^{(r)}$ is shared by all G query heads in group r.

Main Branch. Given the block index set $\mathcal{I}_i^{(r)}$ selected by the Index Branch, the Main Branch attends only to the causally visible tokens in the selected blocks. For any query head $h \in \mathcal{H}_r$, it applies standard scaled dot-product attention restricted to these tokens, using the key-value head associated with GQA group r:

$$\boldsymbol{O}_i^{(h)} = \mathrm{softmax}\left(\frac{\boldsymbol{Q}_i^{(h)} \left(\boldsymbol{K}^{(r)}\left[\mathcal{I}_i^{(r)}\right]\right)^{\top}}{\sqrt{d_h}}\right)\boldsymbol{V}^{(r)}\left[\mathcal{I}_i^{(r)}\right], \tag{8}$$

where $\boldsymbol{Q}_i^{(h)}$ denotes the query vector at position i and query head h, while $\boldsymbol{K}^{(r)}$ and $\boldsymbol{V}^{(r)}$ denote the key and value matrices of the r-th GQA group. The notation $\boldsymbol{K}^{(r)}[\mathcal{I}_i^{(r)}]$ and $\boldsymbol{V}^{(r)}[\mathcal{I}_i^{(r)}]$ denotes gathering the causally visible tokens from the selected blocks. The block index set $\mathcal{I}_i^{(r)}$ is shared by all query heads in $\mathcal{H}_r$, while each head keeps its own query projection. Since the selected blocks contain at most kB_k causally visible tokens, the per-query attention cost is reduced from $O(N)$ to $O(kB_k)$, which is fixed as the sequence length increases.

3.2. Training

The top-k selection in Equation 7 is non-differentiable, so the language-modeling loss cannot train the index Q/K projections $\boldsymbol{W}_q^{\text{idx}}, \boldsymbol{W}_k^{\text{idx}}$ directly. We therefore train the Index Branch with a KL alignment loss and use three mechanisms to stabilise sparse training: **Gradient Detach**, **Indexer Warmup**, and a forced **Local Block**. We describe each component below.

KL Loss. The KL loss gives the Index Branch a direct learning signal by matching its scores to the Main Branch on the selected tokens. Writing $\mathcal{I}_{i,\text{tok}}^{(r)} = (\bigcup_{b \in \mathcal{I}_i^{(r)}} \mathcal{B}_b) \cap \{1, \dots, i\}$ for the causally visible tokens induced by the selected block indices, for each query position i and GQA group r, we define the Index Branch distribution P^{idx} and the Main Branch teacher P over this token index set:

$$P_{i,j}^{\text{idx},(r)} = \frac{\exp(S_{i,j}^{\text{idx},(r)})}{\sum_{u \in \mathcal{I}_{i,\text{tok}}^{(r)}} \exp(S_{i,u}^{\text{idx},(r)})}, \qquad P_{i,j}^{(r)} = \frac{1}{G} \sum_{\ell \in \mathcal{H}_r} \frac{\exp(S_{i,j}^{(\ell)})}{\sum_{u \in \mathcal{I}_{i,\text{tok}}^{(r)}} \exp(S_{i,u}^{(\ell)})}, \qquad j \in \mathcal{I}_{i,\text{tok}}^{(r)}, \tag{9}$$

where $S_{i,j}^{\text{idx},(r)} = (\boldsymbol{Q}^{\text{idx}})_i^{(r)} (\boldsymbol{K}^{\text{idx}})_j^{\top} / \sqrt{d_{\text{idx}}}$ is the token-level index score, and $S_{i,j}^{(\ell)} = \boldsymbol{Q}_i^{(\ell)} (\boldsymbol{K}_j^{(r)})^{\top} / \sqrt{d_h}$ is the Main Branch score for query head $\ell \in \mathcal{H}_r$. The teacher P averages the per-head Main Branch distributions at the probability level. The indexer is then trained to match P, averaged over all query positions and GQA groups:

$$\mathcal{L}_{\text{KL}} = \frac{1}{N H_{kv}} \sum_{i=1}^{N} \sum_{r=1}^{H_{kv}} D_{\text{KL}}\big(\text{stopgrad}(P_{i,\cdot}^{(r)}) \,\|\, P_{i,\cdot}^{\text{idx},(r)}\big), \tag{10}$$

where N is the sequence length, and the teacher distribution $P_{i,\cdot}^{(r)}$ is detached from gradient computation. This auxiliary loss aligns the index distribution with the Main Branch attention pattern, making the subsequent block selection semantically meaningful.

Gradient Detach. To isolate the auxiliary objective from the backbone, we apply stop-gradient to the Index Branch input:

$$\boldsymbol{Q}^{\text{idx}} = \text{stopgrad}(\boldsymbol{X}) \boldsymbol{W}_q^{\text{idx}}, \qquad \boldsymbol{K}^{\text{idx}} = \text{stopgrad}(\boldsymbol{X}) \boldsymbol{W}_k^{\text{idx}}. \tag{11}$$

The teacher P in Equation 9 is detached, so $\mathcal{L}_{\text{KL}}$ leaves the Main Branch projections untouched; Equation 11 further prevents it from reaching the backbone through $\boldsymbol{X}$. Under this rule, $\mathcal{L}_{\text{KL}}$ updates only $\boldsymbol{W}_q^{\text{idx}}$ and $\boldsymbol{W}_k^{\text{idx}}$, making the KL a clean alignment signal for the indexer.

Indexer Warmup. We use a two-stage training schedule to initialise the Index Branch and avoid early random selections. During the first few iterations, the model runs full attention in both branches and trains the newly added index projections with $\mathcal{L}_{\text{KL}}$. After warmup, the model switches to sparse attention, and $\mathcal{L}_{\text{KL}}$ is computed over the top-k selected positions. The same schedule is used when sparsifying a pretrained full-attention checkpoint, which helps align the newly added index projections before they control Main Branch routing.

Local Block. For each query position i and GQA group r, the local block containing i is always selected as part of $\mathcal{I}_i^{(r)}$ during both training and inference. This fixed allocation reserves one block slot and leaves the remaining slots to be chosen by the Index Branch, preventing degenerate selections that omit the query's immediate neighbourhood.

The complete layer-level training procedure is summarised in Algorithm 1.

Algorithm 1 One MSA layer: training forward and the auxiliary KL loss. The layer returns its output and per-layer $\mathcal{L}_{\text{KL}}$; the model loss $\mathcal{L} = \mathcal{L}_{\text{LM}} + \lambda \sum_{\text{layers}} \mathcal{L}_{\text{KL}}$ is assembled by the training loop.

Require: hidden states $\boldsymbol{X} \in \mathbb{R}^{N \times d_{\text{model}}}$; block size B_k, number of selected blocks k.

$\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V} \leftarrow \boldsymbol{X}\boldsymbol{W}_q,\ \boldsymbol{X}\boldsymbol{W}_k,\ \boldsymbol{X}\boldsymbol{W}_v$ // $(N, H_q, d_h), (N, H_{kv}, d_h), (N, H_{kv}, d_h)$

$\boldsymbol{Q}^{\text{idx}}, \boldsymbol{K}^{\text{idx}} \leftarrow \text{stopgrad}(\boldsymbol{X})\boldsymbol{W}_q^{\text{idx}},\ \text{stopgrad}(\boldsymbol{X})\boldsymbol{W}_k^{\text{idx}}$ // $(N, H_{kv}, d_{\text{idx}}), (N, 1, d_{\text{idx}})$; detached

$M^{\text{idx}} \leftarrow \text{BlockMaxPool}(\boldsymbol{Q}^{\text{idx}}, \boldsymbol{K}^{\text{idx}}, B_k)$ // (N, H_{kv}, B); per-group, causal

$\mathcal{I} \leftarrow \text{TopK}(M^{\text{idx}}, k)$ // selected block indices; local block included

$\boldsymbol{O} \leftarrow \text{TopKAttn}(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}, \mathcal{I})$ // (N, H_q, d_h); attends to selected blocks

output $\leftarrow \boldsymbol{O}\boldsymbol{W}_o$ // (N, d_{model})

$\mathcal{L}_{\text{KL}} \leftarrow \text{KLdiv}(\boldsymbol{Q}^{\text{idx}}, \boldsymbol{K}^{\text{idx}}, \text{stopgrad}(\boldsymbol{Q}), \text{stopgrad}(\boldsymbol{K}), \mathcal{I})$ // over tokens induced by $\mathcal{I}$

return output, $\mathcal{L}_{\text{KL}}$

3.3. Computational Complexity

Under the same H_q, H_{kv}, d_h, and sequence length N, the causal attention FLOPs of GQA and MSA are

$$F_{\text{GQA}}(N) = 2H_q d_h N^2, \qquad F_{\text{MSA}}(N) = \underbrace{H_{kv} d_{\text{idx}} N^2}_{\text{Index Branch}} + \underbrace{4H_q d_h N k B_k}_{\text{Main Branch}}. \tag{12}$$

GQA scales its main attention path with the full context length, whereas MSA uses a fixed selection budget kB_k plus a lightweight index computation; the FLOPs gap therefore grows with N when $kB_k \ll N$ and $H_{kv} d_{\text{idx}} \ll H_q d_h$.

4. Kernel Design

This section describes the GPU kernels used in our sparse prefill implementation, including the index TopK kernel, the KV-outer sparse attention forward, and the sparse KL loss backward.

4.1. Index & TopK

Exp-free selection. To efficiently select the top-k KV blocks, the index module ranks the index scores s directly. Since softmax is order-preserving, the relative ordering of scores is preserved ($s_i \le s_j \iff \text{softmax}(s)_i \le \text{softmax}(s)_j$), leaving the top-$k$ indices unchanged. The forward pass, therefore, bypasses the max/exp/sum steps of softmax and passes raw scores directly to selection.

Per-thread register top-k. The block size B_k and selection size k are co-designed with the top-k kernel: a large B_k increases attention arithmetic intensity (Section 4.2), and a small k at this B_k keeps both the per-row candidate block count B and k below the sweet spot of general-purpose top-k kernels, which amortize multi-pass bucketing over large B (radix selection) or scale as $O(B \log^2 B)$ (bitonic sort). We adopt $B_k = 128$, $k = 16$. Each of the warp's 32 lanes streams a 1/32 stride of the input row and maintains a k-element min-heap in shared memory. The heap root is cached in a register, and insertions are performed with deferred writes. Finally, a k-round shuffle merge combines the 32 local TopK results. The shared-memory layout maps each lane to a fixed bank, avoiding conflicts.

Benchmark. We compare against `torch.topk` and the TileLang (Wang et al., 2025) radix-select top-k on an H800 GPU with `fp32` inputs and unsorted outputs; latencies are the median of 50 post-warmup iterations. Table 1 shows that our specialized kernel is fastest in all tested settings, with the largest gains at the deployed setting $k = 16$.

Seq. Len. N	Blocks B	k	`torch`	TileLang	Ours	vs. `torch`	vs. TileLang
128K	1024	16	3970	2864	779	5.1×	3.7×
128K	2048	32	5378	3630	1991	2.7×	1.8×
512K	4096	16	33810	17779	7880	4.3×	2.3×
512K	8192	32	57659	26100	21326	2.7×	1.2×

Table 1 | Top-k latency (μs) for `fp32` inputs of shape (N, B), with rows processed independently. The deployed setting uses $B_k = 128$, $k = 16$, while for reference we also report $k = 32$ with $B_k = 64$. All implementations produce identical index sets.

4.2. Sparse Attention

We revisit the choice of iteration order under sparse prefill with equal query and key/value lengths. Let H_q, H_{kv}, $G = H_q/H_{kv}$, d_h, N, B_k, and k denote the number of query heads, key-value heads, GQA ratio, head dimension, sequence length, KV block size, and number of blocks selected per query. For simplicity, the IO estimates below assume 2-byte elements (bfloat16-sized traffic). Our kernels also support fp8; using fp8 rescales the absolute IO volume but leaves the comparison between Q-outer and KV-outer iteration unchanged.

Iterating queries on the outer loop gives

$$\text{FLOPs} = 4\, H_q\, N\, d_h\, k\, B_k, \tag{13}$$

$$\text{IO} = \underbrace{2 \cdot 2 \cdot H_q\, N\, d_h}_{\text{read}(\boldsymbol{Q})+\text{write}(\boldsymbol{O})} + \underbrace{2 \cdot 2 \cdot H_{kv}\, N\, k\, B_k\, d_h}_{\text{read}(\boldsymbol{K}+\boldsymbol{V})}, \tag{14}$$

hence FLOPs/IO $\approx G$.

Iterating KV blocks on the outer loop and gathering the queries that selected each block requires an intermediate output buffer:

$$\text{FLOPs} = 4\, H_q\, N\, d_h\, k\, B_k, \tag{15}$$

$$\text{IO} = \underbrace{2 \cdot 2 \cdot H_{kv}\, N\, d_h}_{\text{read}(\boldsymbol{K}+\boldsymbol{V})} + \underbrace{2 \cdot 2 \cdot H_q\, N\, k\, d_h}_{\text{read}(\boldsymbol{Q})+\text{write}(\boldsymbol{O}_{\text{buf}})} + \underbrace{2 \cdot H_q\, N\, (k{+}1)\, d_h}_{\text{read}(\boldsymbol{O}_{\text{buf}})+\text{write}(\boldsymbol{O})}, \tag{16}$$

hence FLOPs/IO $\approx \frac{2}{3} B_k$.

Since $\frac{2}{3} B_k \gg G$ in practice, we choose KV-outer iteration with Q gather to maximize arithmetic intensity. The kernel executes as a persistent grid over (kv_block, kv_head) tiles. For each tile, a reverse sparse index from the TopK selection identifies the relevant query positions. These queries are loaded into shared memory via TMA copies, one per query token, dispatched in parallel by the 32 lanes of a warp.

Pre-scheduled tile chunking. A direct one-CTA-per-tile mapping is dominated by sink rows—a single early KV block selected by nearly every query—and the same hotspot pattern can arise on any popular KV block. A GPU scheduler kernel therefore splits each KV tile along its query dimension into chunks of at most $\sim 2kB_k$ queries each, fanning hot tiles across many CTAs that share the same **K**/**V** load. Because each query's k partials are now produced by k CTAs, the scheduler also preassigns each (query, chunk) pair a slot $s \in [0, k)$ in $\mathbf{O}_{\text{buf}}$—packed with the query index i into a 32-bit handle—so the attention kernel writes its partial to the preassigned offset without atomics. The combine kernel reads a per-query slot count to know how many partials to merge.

Two-phase forward. The KV-outer split forbids inline softmax normalization since each query's k partials are produced by k different CTAs. The forward is therefore split into two kernels separated by HBM buffers $\mathbf{O}_{\text{buf}} \in \mathbb{R}^{k\times n\times H_q\times d}$ (locally normalized partial outputs) and $\text{LSE}_{\text{buf}} \in \mathbb{R}^{k\times n\times H_q}$ (per-partial logsumexps). The attention kernel runs the worklist above and writes each partial to its preassigned slot. The combine kernel reads the valid slots of each query, computes $a = \max_s \text{LSE}_s$ and $\text{LSE}[i,h] = a + \log \sum_s \exp(\text{LSE}_s - a)$, then forms normalized split-K weights $w_s = \exp(\text{LSE}_s - \text{LSE}[i,h])$. It outputs $\mathbf{O}[i,h] = \sum_s w_s \, \mathbf{O}_{\text{buf}}[s,i,h]$ together with the final $\text{LSE}[i,h]$. The two kernels use Programmatic Dependent Launch to hide the inter-kernel launch latency.

Query concatenation. KV-outer iteration often associates each KV tile with only a few to a few tens of query positions. Processing these positions one at a time would under-fill the score MMA: with $G = 16$, a single query position contributes only G query heads, yielding an MMA M dimension of only 16. Under Q-outer iteration, query positions cannot be concatenated along the sequence dimension because they generally select different KV subsets. Under KV-outer iteration, however, all gathered positions for a given tile share the same KV operands. The kernel, therefore, packs $\lceil 128/G \rceil$ query positions together with their G associated query heads, all under the same KV head, into a 128×128 score MMA.

4.3. Sparse KL Loss

LSE fusion. In our initial implementation, we utilized a dedicated kernel to compute the KL divergence forward pass, storing LSE_{main} and LSE_{idx} to facilitate backpropagation. However, since the KL loss only affects the backward gradient, we optimize this by emitting these LSE values directly to global memory during the main pass, allowing us to skip the KL loss forward pass entirely. Additionally, during the index branch computation, we save the per-block LSEs and perform a reduction over the top-k blocks to obtain LSE_{idx}. The backward kernel then loads these scalars directly into the softmax, eliminating the redundant forward computation.

Dynamic load balancing. Per-tile work varies by orders of magnitude under variable-length sequences and data-dependent sparsity. The kernel runs as a persistent grid in which CTAs claim work through a global atomic counter; each tile is partitioned along its gathered-query dimension into sub-tiles whose count scales with the per-tile query count, subject to a minimum sub-tile granularity that amortizes per-sub-tile overhead.

5. Experiment

This section reports two 109B-scale experiments used to validate the final MSA design on a native multimodal model trained on a mixture of text and image/video data. The first trains a native MSA model from scratch, which we denote as **MSA-PT**. The second starts from a Full-Attention checkpoint and continues pretraining after replacing dense attention with MSA, which we denote as **MSA-CPT**. Both models use the same architecture family as the Full-Attention baseline, but replace dense attention with the MSA layer.

5.1. Setup

Model Structure. All models use the same 41-layer MoE backbone, with approximately 109B total parameters and 6B activated parameters per token. The first three layers are dense layers, and the remaining 38 layers are MoE layers. The model uses a 200K-token vocabulary and hidden size

$d_{\text{model}} = 3072$. Each attention module uses MSA with 64 query heads, 4 KV heads, head dimension 128, and RoPE dimension 64. Each MoE layer uses 128 routed experts, 1 shared expert, and top-4 routed expert selection. During sparse training and evaluation, both MSA models use block size $B_k = 128$ and keep $k = 16$ key-value blocks per query and GQA group.

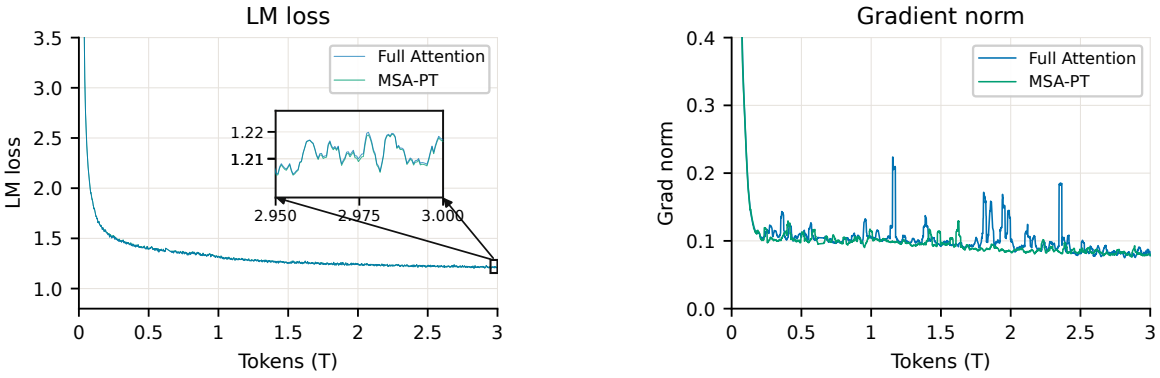
Training Budget. All models are trained under a total budget of 3T tokens. MSA-PT is trained from scratch: after a 40B-token indexer warmup, it remains in sparse training for the rest of pretraining. MSA-CPT starts from a GQA Full-Attention checkpoint trained on 2.6T tokens. We then replace dense attention with MSA and continue training for 400B tokens: the first 40B tokens are used for indexer warmup, followed by sparse continued pretraining.

Evaluations. We evaluate Full, MSA-PT, and MSA-CPT on the same pretraining evaluation suite using matched checkpoints under the same training budget. For general reasoning and question answering, we use MMLU (Hendrycks et al., 2021), MMLU-Pro (Wang et al., 2024a), BBH (Suzgun et al., 2022), GPQA Hard (Rein et al., 2023), ARC Challenge (Clark et al., 2018), TriviaQA (Joshi et al., 2017), and WinoGrande (Sakaguchi et al., 2020). For math and code, we use GSM8K (Cobbe et al., 2021), MGSM (Shi et al., 2022), MathVista (Lu et al., 2024), OlymMATH (Sun et al., 2025), HumanEval (Chen et al., 2021), EvalPlus (Liu et al., 2023), BigCodeBench (Zhuo et al., 2025), and MultiPL-E MBPP (Cassano et al., 2023). We also evaluate multimodal capability: image benchmarks include AI2D (Kembhavi et al., 2016), ChartQA (Masry et al., 2022), MMMU (Yue et al., 2024), OCRBench v2 (Fu et al., 2025), CharXiv (Wang et al., 2024b), VisualWebBench (Liu et al., 2024), and CVBench (Tong et al., 2024), while video benchmarks include EgoSchema (Mangalam et al., 2023), LongVideoBench (Wu et al., 2024), MLVU (Zhou et al., 2025), MMVU (Zhao et al., 2025b), VideoMME (Fu et al., 2024), and TemporalBench (Cai et al., 2024). For long-context evaluation, we use RULER (Hsieh et al., 2024) and HELMET (Yen et al., 2025). We additionally report perplexity on downstream agent tasks, including τ^2-bench (Barres et al., 2025), TheAgentCompany (Xu et al., 2024), Humanity's Last Exam (Phan et al., 2025), and SWE-bench (Jimenez et al., 2024).

5.2. Training Dynamics

Figure 2 compares native sparse pretraining with the matched full-attention run. Over the 3T-token training process, the two LM-loss curves are nearly indistinguishable, showing that MSA does not introduce noticeable optimization degradation relative to full attention. The gradient-norm curves also remain within the same range throughout training, suggesting that MSA does not lead to abnormal gradient fluctuations or training instability. These results indicate that training a sparse attention model is as stable as training the full-attention baseline at a large scale.

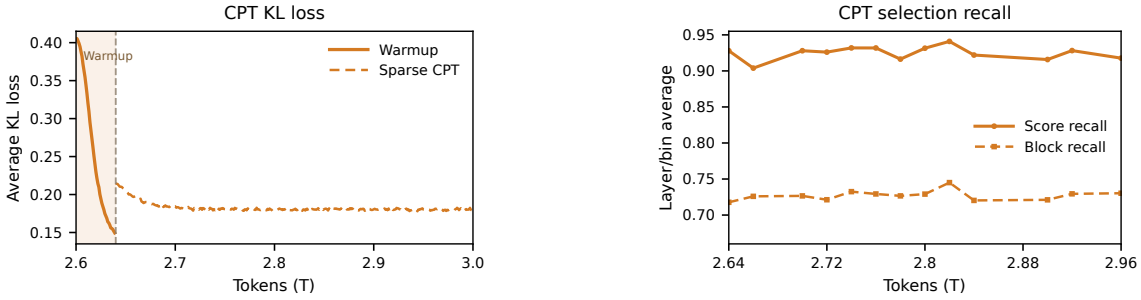
Figure 3 illustrates the transition from a trained full-attention checkpoint to sparse continued pretraining. The indexer-warmup stage rapidly reduces the KL loss before sparse attention is enabled. After switching to sparse CPT, the KL loss remains low. For each query and GQA head, let $\mathcal{I}^\star$ be the corresponding Top-k block set induced by the Main Branch scores and let $\widehat{\mathcal{I}}$ be the Index Branch selection. Block recall is $|\mathcal{I}^\star \cap \widehat{\mathcal{I}}|/|\mathcal{I}^\star|$, while score recall is $\sum_{b \in \mathcal{I}^\star \cap \widehat{\mathcal{I}}} P_b / \sum_{b \in \mathcal{I}^\star} P_b$, where P_b is the Main Branch attention probability summed over tokens in block b. The block recall stays favorable, indicating reliable recovery of important blocks. The higher score recall further shows that the retrieved blocks account for most of the Main Branch attention mass. Together, these dynamics show that warmup provides a clean conversion phase and that the CPT indexer remains well aligned during sparse continued pretraining.



(a) LM loss.

(b) Gradient norm.

Figure 2 | Pretraining dynamics for the experiment model. LM loss and gradient norm are shown for Full Attention and MSA-PT over 3T training tokens. The inset in (a) zooms in on the final 50B-token window, where the two LM-loss curves remain nearly overlapping.



(a) KL loss.

(b) Selection recall.

Figure 3 | Sparse continued-pretraining dynamics. (a) Average KL loss during MSA-CPT. The solid segment denotes indexer warmup, and the dashed segment denotes sparse continued pretraining; the vertical dashed line marks the switch between the two stages. (b) Average block recall and score recall of the MSA-CPT indexer during sparse continued pretraining.

5.3. Main Results

Table 2 compares Full, MSA-PT, and MSA-CPT on a representative set of pretraining evaluations. Both sparse models remain broadly competitive with the Full-Attention baseline, indicating that replacing dense attention with MSA does not substantially degrade the model's general language, reasoning, multimodal, or agent-oriented perplexity profile. The two training routes show different strengths. MSA-PT, which learns the sparse pattern throughout pretraining, obtains the strongest results on many math, image, video, and long-context retrieval benchmarks, suggesting that native sparse pretraining can adapt the model representations to the sparse attention pattern. MSA-CPT is more conservative: it preserves much of the Full-Attention checkpoint behavior and remains close on most text, code, and PPL evaluations, making it a practical conversion route when a trained dense checkpoint is already available. The remaining gaps are benchmark-dependent rather than concentrated in a single capability area.

Table 2 | Representative evaluation results under the 3T-token training budget. Full denotes the Full-Attention baseline, MSA-PT denotes from-scratch sparse pretraining, and MSA-CPT denotes sparse continued pretraining. Best per-row results are bolded; lower is better for PPL and higher is better otherwise.

Group	Benchmark	Full	MSA-PT	MSA-CPT
General	MMLU	67.0	**67.2**	66.8
	MMLU-Pro	38.5	38.8	**39.1**
	BBH	**67.7**	66.6	66.1
	GPQA Hard	25.9	**26.3**	**26.3**
	ARC Challenge	82.7	82.5	**82.9**
	TriviaQA	66.0	65.5	**67.7**
	WinoGrande	58.3	60.9	**62.0**
Math	GSM8K	76.2	**77.7**	73.7
	MGSM	44.1	**46.0**	44.2
	MathVista	43.8	**46.8**	44.5
	OlymMATH Easy P@100	23.0	**26.0**	22.0
Code	HumanEval	61.0	**64.0**	57.9
	EvalPlus	59.4	**61.8**	60.0
	BigCodeBench	44.8	44.0	**45.7**
	MultiPL-E MBPP P@10	**82.1**	81.6	81.1
Retrieval	RULER-8K	79.8	**84.2**	77.2
	RULER-32K	75.0	**77.5**	75.7
Image	AI2D	68.3	**70.6**	67.3
	ChartQA	75.0	**75.4**	71.4
	MMMU	**46.8**	45.9	44.5
	OCRBench v2	55.0	**55.7**	54.3
	CharXiv	37.55	**41.55**	37.15
	VisualWebBench	55.6	**68.4**	59.4
	CVBench	57.0	**59.7**	58.8
Video	EgoSchema	29.6	**37.6**	25.8
	LongVideoBench	38.5	**41.8**	38.9
	MLVU	44.14	**46.94**	43.68
	MMVU	45.8	**47.5**	45.8
	VideoMME	41.11	**45.48**	39.65
	TemporalBench	49.4	**53.4**	50.6
PPL ↓	TAU2	1.155	**1.148**	1.150
	AgentCompany	1.248	1.249	**1.247**
	HLE	**1.275**	1.278	**1.275**
	SWE	**1.216**	1.218	**1.216**

To evaluate whether MSA remains effective after long-context scaling, we conduct an additional extension experiment on the MSA-CPT model. Starting from the sparse continued-pretraining checkpoint, we run approximately 140B tokens of long-context training and then evaluate on HELMET and RULER. The results are reported in Table 3. After the extension stage, MSA-CPT remains close to the Full-Attention baseline. Since each query and GQA group still attends to only $kB_k = 16 \times 128 = 2{,}048$ key-value tokens, these results indicate that MSA can preserve long-context capability under a highly tight attention budget.

Additional ablations supporting these design choices are provided in the appendix. In particular, Section B studies the training recipe for the Index Branch, including gradient sources, KL-gradient detachment, warmup, and the comparison with a sliding-window sparse baseline. Section C further examines architectural choices such as block size, forced sink, local selection, and the Index Branch

value head. These ablations provide the empirical basis for the final MSA design used in the main experiments.

Table 3 | Long-context extension results for MSA-CPT on HELMET and RULER. Δ reports the difference between MSA-CPT and the Full-Attention baseline. The "Overall" score is averaged across the fine-grained subtasks. Higher is better for all metrics.

Benchmark	Subset	Full	MSA-CPT	Δ
HELMET-128K	Overall	**46.53**	45.93	-0.60
	ICL	70.40	**72.80**	+2.40
	Rerank/RAG	**34.60**	32.50	-2.10
RULER-128K	Overall	72.00	**72.12**	+0.12
	CWE/FWE	**46.35**	45.00	-1.35
	MK/MQ/MV	96.63	**98.87**	+2.24
	QA1/QA2	**47.80**	46.80	-1.00
	VT	**97.80**	96.80	-1.00

5.4. Efficiency

We instantiate the complexity analysis in Section 3.3 on our experimental model configuration and report both theoretical attention-FLOPs reduction and measured runtime speedup. Dense GQA and MSA use the same query head count, key-value head count, head dimension, and context length; the only difference is that dense GQA attends to the full context, whereas MSA performs index selection followed by sparse attention over a fixed KV budget. In our setting, MSA uses $B_k = 128$ and $k = 16$, corresponding to a selected budget of $kB_k = 2{,}048$ tokens per query.

As shown in Figure 4, MSA reduces per-token attention FLOPs substantially relative to GQA in our setting, with the reduction increasing at longer contexts. At 1M tokens, the FLOPs reduction reaches 28.4× under the same head configuration. The measured runtime speedup follows the same scaling trend but is not expected to match the FLOPs reduction exactly. Sparse attention introduces index construction, top-k selection, reverse-index materialization, query gathering, and load-balancing overheads, and its memory access pattern is less regular than dense attention. Therefore, the runtime speedup is smaller than the theoretical FLOPs reduction, but it increases with context length as the dense baseline continues to scale with the full sequence while MSA keeps the main attention budget fixed.

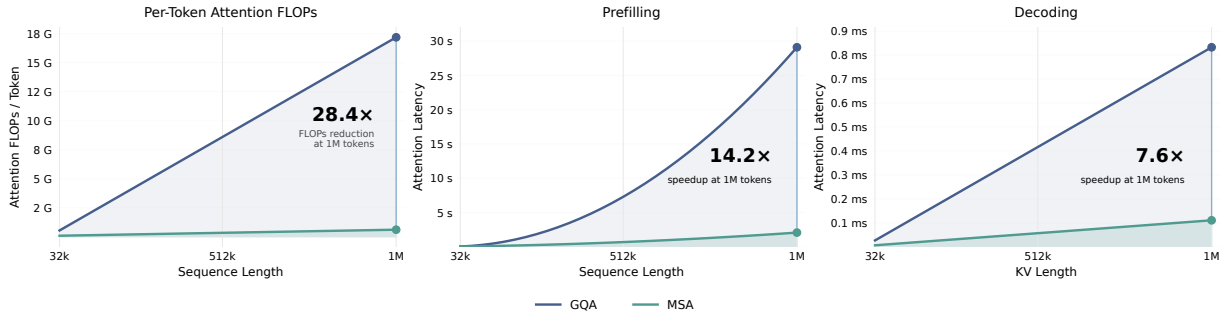


Figure 4 | Efficiency comparison between GQA and MSA under the shared experimental model configuration. The left subfigure reports the theoretical per-token attention-FLOPs. The middle and right subfigures report the measured implementation speedups for prefill and decode, respectively. All tests are conducted with 64 query heads, 4 key-value heads, and a head dimension of 128. MSA uses $B_k = 128$ and $k = 16$, corresponding to a selected budget of 2,048 tokens per query.

6. Related Works

Long-context efficiency has motivated a large body of work on efficient attention, which can be broadly grouped into two directions: replacing dense softmax attention with cheaper linear or recurrent alternatives, and retaining softmax attention while restricting its receptive field. Linear attention (Choromanski et al., 2021; Katharopoulos et al., 2020) replaces the softmax kernel with a linear-complexity surrogate, while state-space models such as Mamba (Gu and Dao, 2023) replace attention with a selective recurrence over hidden states. Hybrid stacks (MiniMax, 2025a,b) interleave linear blocks with full-attention blocks, reducing the number of quadratic layers while preserving part of the exact-softmax capacity. Fixed-pattern attention keeps softmax attention but imposes a predefined support, including local windows, global tokens (Beltagy et al., 2020; Zaheer et al., 2020), and attention sinks with a sliding window (Xiao et al., 2024b). These approaches reduce long-context cost either by replacing dense attention in part or in full, or by using a content-agnostic attention pattern.

Beyond fixed sparse patterns, adaptive sparse attention makes the attended support depend on the input. Existing methods differ mainly in when this support is constructed and whether the selector is trained as part of the model. Inference-time sparsification operates on a pretrained Full-Attention backbone and constructs sparse supports only during serving. H2O (Zhang et al., 2023) and SnapKV (Li et al., 2024) prune the KV cache during decoding using accumulated attention statistics, Quest (Tang et al., 2024) performs page-level importance estimation per query, MInference (Jiang et al., 2024) and FlexPrefill (Lai et al., 2025) dispatch per-head sparse kernels at prefill, and InfLLM (Xiao et al., 2024a) maintains attention sinks, a local context window, and retrievable chunks. These methods inherit the training cost of Full Attention and leave at least one inference phase near Full-Attention speed. Natively trained sparse-attention designs train the indexer during pretraining and form the closest prior work to MSA. NSA (Yuan et al., 2025) targets MQA/MHA backbones with three parallel branches: compressed attention for coarse global context, selected attention over fine-grained blocks, and a sliding window for local context. InfLLM-V2 (Zhao et al., 2025a) achieves zero-shot dense-to-sparse switching by unifying parameter-free block selection with a local sliding window. MoBA (Lu et al., 2025) also operates on GQA but uses very large KV blocks scored by block-averaged keys, and trains its indexer only through the language-modeling gradient. DSA (DeepSeek-AI et al., 2025) sits on top of MLA in its MQA mode: a multi-head ReLU-based lightning indexer scores tokens individually, all query heads share a single Top-k index, and selection is token-level. MSA differs from this neighborhood along two axes that are taken up together: per-GQA-group Top-k sharing combined with block-level selection, which gives multi-group block-granular retrieval while keeping KV reads contiguous.

Efficient kernels are essential for sparse attention to translate theoretical FLOP reduction into wall-clock speedups. FlashAttention (Dao et al., 2022) and FlashAttention-2 (Dao, 2024) introduced IO-aware tiled softmax attention, and FlashDecoding (Dao et al., 2023) extended this to memory-bound decoding. Open-source block-sparse kernels such as Flash-Sparse-Attention (Yan et al., 2025) and FlashMoBA (Xiao et al., 2025) have made block-sparse variants of this recurrence available. MSA's kernel, described in section 4, reuses the FlashAttention algorithmic skeleton with a loop ordering tuned to the GQA-native, block-granular access pattern MSA produces.

7. Conclusion

We introduced MSA, a sparse-attention mechanism co-designed with Grouped-Query Attention. The architecture attaches a lightweight Index Branch to a standard GQA layer: each GQA group independently selects a small set of key-value blocks through a block-level dot-product indexer, and

the Main Branch performs softmax attention restricted to the selected blocks. The Index Branch is a pure selector and is trained by a KL alignment loss against the Main Branch under a two-stage warmup schedule and a stop-gradient on the index input that confines the auxiliary loss to the index projections. At the 109B-MoE scale, MSA preserves the capability of a GQA Full-Attention baseline across most pretraining and agentic benchmarks while reducing per-token attention compute by 28.4× at 1M context, the regime in which long-context inference becomes the binding deployment constraint.

Outlook. MSA's core decisions—per-GQA-group independent selection, block-level granularity, and an indexer trained with a KL alignment objective—compose with the GQA backbone shared by most current open-source frontier models, so the recipe should transfer with little modification. Two directions are natural next steps: closing the residual long-context retrieval gap, either through longer sparse training, a larger selection budget at inference time, or a richer indexer scoring function; and extending the same selector-only design to settings beyond pretraining, including reinforcement-learning post-training and agentic deployment, where long-context cost is the dominant operational constraint.

References

Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. GQA: Training generalized multi-query transformer models from multi-head checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

Anthropic. Claude Opus 4.6 and Sonnet 4.6 model card. https://www.anthropic.com/news/claude-4-6, 2025.

Victor Barres, Honghua Dong, Soham Ray, Xujie Si, and Karthik Narasimhan. τ^2-bench: Evaluating conversational agents in a dual-control environment. *arXiv preprint arXiv:2506.07982*, 2025.

Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.

Mu Cai, Reuben Tan, Jianrui Zhang, Bocheng Zou, Kai Zhang, Feng Yao, Fangrui Zhu, Jing Gu, Yiwu Zhong, Yuzhang Shang, Yao Dou, Jaden Park, Jianfeng Gao, Yong Jae Lee, and Jianwei Yang. Temporalbench: Benchmarking fine-grained temporal understanding for multimodal video models, 2024. URL https://arxiv.org/abs/2410.10818.

Federico Cassano, John Gouwar, Daniel Nguyen, Sydney Nguyen, Luna Phipps-Costin, Donald Pinckney, Ming-Ho Yee, Yangtian Zi, Carolyn Jane Anderson, Molly Q. Feldman, Arjun Guha, Michael Greenberg, and Abhinav Jangda. MultiPL-E: A scalable and polyglot approach to benchmarking neural code generation. *IEEE Transactions on Software Engineering*, 49(7):3675–3691, 2023.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish,

Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code, 2021. URL https://arxiv.org/abs/2107.03374.

Krzysztof Choromanski, Valerii Likhosherstov, David Dohan, Xingyou Song, Andreea Gane, Tamás Sarlós, Peter Hawkins, Jared Davis, Afroz Mohiuddin, Lukasz Kaiser, David Belanger, Lucy Colwell, and Adrian Weller. Rethinking attention with performers. In *International Conference on Learning Representations (ICLR)*, 2021.

Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try arc, the ai2 reasoning challenge, 2018. URL https://arxiv.org/abs/1803.05457.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems, 2021. URL https://arxiv.org/abs/2110.14168.

Tri Dao. FlashAttention-2: Faster attention with better parallelism and work partitioning. In *International Conference on Learning Representations (ICLR)*, 2024.

Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*, 2022.

Tri Dao, Daniel Haziza, Francisco Massa, and Grigory Sizov. Flash-decoding for long-context inference, 2023. https://crfm.stanford.edu/2023/10/12/flashdecoding.html.

DeepSeek-AI. DeepSeek-V4: Towards highly efficient million-token context intelligence, 2026. Technical report (preview).

DeepSeek-AI, Aixin Liu, Aoxue Mei, Bangcai Lin, Bing Xue, Bingxuan Wang, Bingzheng Xu, Bochao Wu, Bowei Zhang, Chaofan Lin, Chen Dong, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenhao Xu, Chong Ruan, Damai Dai, Daya Guo, Dejian Yang, Deli Chen, Erhang Li, Fangqi Zhou, Fangyun Lin, Fucong Dai, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Hanwei Xu, Hao Li, Haofen Liang, Haoran Wei, Haowei Zhang, Haowen Luo, Haozhe Ji, Honghui Ding, Hongxuan Tang, Huanqi Cao, Huazuo Gao, Hui Qu, Hui Zeng, Jialiang Huang, Jiashi Li, Jiaxin Xu, Jiewen Hu, Jingchang Chen, Jingting Xiang, Jingyang Yuan, Jingyuan Cheng, Jinhua Zhu, Jun Ran, Junguang Jiang, Junjie Qiu, Junlong Li, Junxiao Song, Kai Dong, Kaige Gao, Kang Guan, Kexin Huang, Kexing Zhou, Kezhao Huang, Kuai Yu, Lean Wang, Lecong Zhang, Lei Wang, Liang Zhao, Liangsheng Yin, Lihua Guo, Lingxiao Luo, Linwang Ma, Litong Wang, Liyue Zhang, M. S. Di, M. Y Xu, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingxu Zhou, Panpan Huang, Peixin Cong, Peiyi Wang, Qiancheng Wang, Qihao Zhu, Qingyang Li, Qinyu Chen, Qiushi Du, Ruiling Xu, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, Runqiu Yin, Runxin Xu, Ruomeng Shen, Ruoyu Zhang, S. H. Liu, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shaofei Cai, Shaoyuan Chen, Shengding Hu, Shengyu Liu, Shiqiang Hu, Shirong Ma, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shuting Pan, Songyang Zhou, Tao Ni, Tao Yun, Tian Pei, Tian Ye, Tianyuan Yue, Wangding Zeng, Wen Liu, Wenfeng Liang, Wenjie Pang, Wenjing Luo, Wenjun Gao, Wentao Zhang, Xi Gao, Xiangwen Wang, Xiao Bi, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaokang Zhang, Xiaotao Nie, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xingkai Yu, Xingyou Li, Xinyu Yang, Xinyuan Li, Xu Chen, Xuecheng Su, Xuehai Pan, Xuheng Lin, Xuwei Fu, Y. Q. Wang, Yang Zhang, Yanhong Xu, Yanru Ma, Yao Li, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Qian, Yi Yu, Yichao Zhang, Yifan Ding, Yifan Shi, Yiliang Xiong, Ying He, Ying Zhou, Yinmin Zhong, Yishi Piao, Yisong Wang, Yixiao Chen, Yixuan

Tan, Yixuan Wei, Yiyang Ma, Yiyuan Liu, Yonglun Yang, Yongqiang Guo, Yongtong Wu, Yu Wu, Yuan Cheng, Yuan Ou, Yuanfan Xu, Yuduan Wang, Yue Gong, Yuhan Wu, Yuheng Zou, Yukun Li, Yunfan Xiong, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Z. F. Wu, Z. Z. Ren, Zehua Zhao, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhibin Gou, Zhicheng Ma, Zhigang Yan, Zhihong Shao, Zhixian Huang, Zhiyu Wu, Zhuoshu Li, Zhuping Zhang, Zian Xu, Zihao Wang, Zihui Gu, Zijia Zhu, Zilin Li, Zipeng Zhang, Ziwei Xie, Ziyi Gao, Zizheng Pan, Zongqing Yao, Bei Feng, Hui Li, J. L. Cai, Jiaqi Ni, Lei Xu, Meng Li, Ning Tian, R. J. Chen, R. L. Jin, S. S. Li, Shuang Zhou, Tianyu Sun, X. Q. Li, Xiangyue Jin, Xiaojin Shen, Xiaosha Chen, Xinnan Song, Xinyi Zhou, Y. X. Zhu, Yanping Huang, Yaohui Li, Yi Zheng, Yuchen Zhu, Yunxian Ma, Zhen Huang, Zhipeng Xu, Zhongyu Zhang, Dongjie Ji, Jian Liang, Jianzhong Guo, Jin Chen, Leyi Xia, Miaojun Wang, Mingming Li, Peng Zhang, Ruyi Chen, Shangmian Sun, Shaoqing Wu, Shengfeng Ye, T. Wang, W. L. Xiao, Wei An, Xianzu Wang, Xiaowen Sun, Xiaoxiang Wang, Ying Tang, Yukun Zha, Zekai Zhang, Zhe Ju, Zhen Zhang, and Zihua Qu. Deepseek-v3.2: Pushing the frontier of open large language models, 2025. URL https://arxiv.org/abs/2512.02556.

Chaoyou Fu, Yuhan Dai, Yongdong Luo, Lei Li, Shuhuai Ren, Renrui Zhang, Zihan Wang, Chenyu Zhou, Yunhang Shen, Mengdan Zhang, et al. Video-MME: The first-ever comprehensive evaluation benchmark of multi-modal LLMs in video analysis. *arXiv preprint arXiv:2405.21075*, 2024.

Ling Fu, Zhebin Kuang, Jiajun Song, Mingxin Huang, Biao Yang, Yuzhe Li, Linghao Zhu, Qidi Luo, Xinyu Wang, Hao Lu, Zhang Li, Guozhi Tang, Bin Shan, Chunhui Lin, Qi Liu, Binghong Wu, Hao Feng, Hao Liu, Can Huang, Jingqun Tang, Wei Chen, Lianwen Jin, Yuliang Liu, and Xiang Bai. Ocrbench v2: An improved benchmark for evaluating large multimodal models on visual text localization and reasoning, 2025. URL https://arxiv.org/abs/2501.00321.

Google DeepMind. Gemini 3.1 pro. https://deepmind.google/technologies/gemini/, 2025.

Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.

Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In *International Conference on Learning Representations*, 2021. URL https://openreview.net/forum?id=d7KBjmI3GmQ.

Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekesh, Fei Jia, and Boris Ginsburg. RULER: What's the real context size of your long-context language models? In *First Conference on Language Modeling*, 2024. URL https://openreview.net/forum?id=kIoBbc76Sy.

Huiqiang Jiang, Yucheng Li, Chengruidong Zhang, Qianhui Wu, Xufang Luo, Surin Ahn, Zhenhua Han, Amir H. Abdi, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. MInference 1.0: Accelerating pre-filling for long-context LLMs via dynamic sparse attention. In *Advances in Neural Information Processing Systems*, 2024.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.

Mandar Joshi, Eunsol Choi, Daniel Weld, and Luke Zettlemoyer. TriviaQA: A large scale distantly supervised challenge dataset for reading comprehension. In Regina Barzilay and Min-Yen Kan, editors, *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume*

1: Long Papers), pages 1601–1611, Vancouver, Canada, July 2017. Association for Computational Linguistics. doi: 10.18653/v1/P17-1147. URL https://aclanthology.org/P17-1147/.

Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are RNNs: Fast autoregressive transformers with linear attention. In *International Conference on Machine Learning (ICML)*, 2020.

Aniruddha Kembhavi, Michael Salvato, Eric Kolve, Minjoon Seo, Hannaneh Hajishirzi, and Ali Farhadi. A diagram is worth a dozen images. In *European Conference on Computer Vision (ECCV)*, 2016.

Xunhao Lai, Jianqiao Lu, Yao Luo, Yiyuan Ma, and Xun Zhou. Flexprefill: A context-aware sparse attention mechanism for efficient long-sequence inference. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net, 2025. URL https://openreview.net/forum?id=OfjIlbelrT.

Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. SnapKV: LLM knows what you are looking for before generation. In *Advances in Neural Information Processing Systems*, 2024.

Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and LINGMING ZHANG. Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, editors, *Advances in Neural Information Processing Systems*, volume 36, pages 21558–21572. Curran Associates, Inc., 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/file/43e9d647ccd3e4b7b5baab53f0368686-Paper-Conference.pdf.

Junpeng Liu, Yifan Song, Bill Yuchen Lin, Wai Lam, Graham Neubig, Yuanzhi Li, and Xiang Yue. Visualwebbench: How far have multimodal LLMs evolved in web page understanding and grounding? In *First Conference on Language Modeling*, 2024. URL https://openreview.net/forum?id=egVSgtJJAx.

Enzhe Lu, Zhejun Jiang, Jingyuan Liu, Yulun Du, Tao Jiang, Chao Hong, Shaowei Liu, Weiran He, Enming Yuan, Yuzhi Wang, Zhiqi Huang, Huan Yuan, Suting Xu, Xinran Xu, Guokun Lai, Yanru Chen, Huabin Zheng, Junjie Yan, Jianlin Su, Yuxin Wu, Neo Y. Zhang, Zhilin Yang, Xinyu Zhou, Mingxing Zhang, and Jiezhong Qiu. Moba: Mixture of block attention for long-context llms, 2025. URL https://arxiv.org/abs/2502.13189.

Pan Lu, Hritik Bansal, Tony Xia, Jiacheng Liu, Chunyuan Li, Hannaneh Hajishirzi, Hao Cheng, Kai-Wei Chang, Michel Galley, and Jianfeng Gao. Mathvista: Evaluating mathematical reasoning of foundation models in visual contexts. In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=KUNzEQMWU7.

Karttikeya Mangalam, Raiymbek Akshulakov, and Jitendra Malik. EgoSchema: A diagnostic benchmark for very long-form video language understanding. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2023.

Ahmed Masry, Do Xuan Long, Jia Qing Tan, Shafiq Joty, and Enamul Hoque. ChartQA: A benchmark for question answering about charts with visual and logical reasoning. In Smaranda Muresan, Preslav Nakov, and Aline Villavicencio, editors, *Findings of the Association for Computational Linguistics: ACL 2022*, pages 2263–2279, Dublin, Ireland, May 2022. Association for Computational Linguistics. doi: 10.18653/v1/2022.findings-acl.177. URL https://aclanthology.org/2022.findings-acl.177/.

MiMo, Bangjun Xiao, Bingquan Xia, Bo Yang, Bofei Gao, Bowen Shen, Chen Zhang, Chenhong He, Chiheng Lou, Fuli Luo, Gang Wang, Gang Xie, Hailin Zhang, Hanglong Lv, Hanyu Li, Heyu Chen, Hongshen Xu, Houbin Zhang, Huaqiu Liu, Jiangshan Duo, Jianyu Wei, Jiebao Xiao, Jinhao Dong, Jun Shi, Junhao Hu, Kainan Bao, Kang Zhou, Lei Li, Liang Zhao, Linghao Zhang, Peidian Li, Qianli Chen, Shaohui Liu, Shihua Yu, Shijie Cao, Shimao Chen, Shouqiu Yu, Shuo Liu, Tianling Zhou, Weijiang Su, Weikun Wang, Wenhan Ma, Xiangwei Deng, Bohan Mao, Bowen Ye, Can Cai, Chenghua Wang, Chengxuan Zhu, Chong Ma, Chun Chen, Chunan Li, Dawei Zhu, Deshan Xiao, Dong Zhang, Duo Zhang, Fangyue Liu, Feiyu Yang, Fengyuan Shi, Guoan Wang, Hao Tian, Hao Wu, Heng Qu, Hongfei Yi, Hongxu An, Hongyi Guan, Xing Zhang, Yifan Song, Yihan Yan, Yihao Zhao, Yingchun Lai, Yizhao Gao, Yu Cheng, Yuanyuan Tian, Yudong Wang, Zhen Tang, Zhengju Tang, Zhengtao Wen, Zhichao Song, Zhixian Zheng, Zihan Jiang, Jian Wen, Jiarui Sun, Jiawei Li, Jinlong Xue, Jun Xia, Kai Fang, Menghang Zhu, Nuo Chen, Qian Tu, Qihao Zhang, Qiying Wang, Rang Li, Rui Ma, Shaolei Zhang, Shengfan Wang, Shicheng Li, Shuhao Gu, Shuhuai Ren, Sirui Deng, Tao Guo, Tianyang Lu, Weiji Zhuang, Weikang Zhang, Weimin Xiong, Wenshan Huang, Wenyu Yang, Xin Zhang, Xing Yong, Xu Wang, Xueyang Xie, Yilin Jiang, Yixin Yang, Yongzhe He, Yu Tu, Yuanliang Dong, Yuchen Liu, Yue Ma, Yue Yu, Yuxing Xiang, Zhaojun Huang, Zhenru Lin, Zhipeng Xu, Zhiyang Chen, Zhonghua Deng, Zihan Zhang, and Zihao Yue. Mimo-v2-flash technical report, 2026. URL https://arxiv.org/abs/2601.02780.

MiniMax. MiniMax-01: Scaling foundation models with lightning attention. *arXiv preprint arXiv:2501.08313*, 2025a.

MiniMax. MiniMax-M1: Scaling test-time compute efficiently with lightning attention. *arXiv preprint arXiv:2506.13585*, 2025b.

Moonshot AI. Kimi K2.6: Open agentic foundation model. https://moonshotai.github.io/Kimi-K2/, 2026.

OpenAI. Introducing GPT-5. https://openai.com/gpt-5/, 2025.

OpenAI, :, Sandhini Agarwal, Lama Ahmad, Jason Ai, Sam Altman, Andy Applebaum, Edwin Arbus, Rahul K. Arora, Yu Bai, Bowen Baker, Haiming Bao, Boaz Barak, Ally Bennett, Tyler Bertao, Nivedita Brett, Eugene Brevdo, Greg Brockman, Sebastien Bubeck, Che Chang, Kai Chen, Mark Chen, Enoch Cheung, Aidan Clark, Dan Cook, Marat Dukhan, Casey Dvorak, Kevin Fives, Vlad Fomenko, Timur Garipov, Kristian Georgiev, Mia Glaese, Tarun Gogineni, Adam Goucher, Lukas Gross, Katia Gil Guzman, John Hallman, Jackie Hehir, Johannes Heidecke, Alec Helyar, Haitang Hu, Romain Huet, Jacob Huh, Saachi Jain, Zach Johnson, Chris Koch, Irina Kofman, Dominik Kundel, Jason Kwon, Volodymyr Kyrylov, Elaine Ya Le, Guillaume Leclerc, James Park Lennon, Scott Lessans, Mario Lezcano-Casado, Yuanzhi Li, Zhuohan Li, Ji Lin, Jordan Liss, Lily, Liu, Jiancheng Liu, Kevin Lu, Chris Lu, Zoran Martinovic, Lindsay McCallum, Josh McGrath, Scott McKinney, Aidan McLaughlin, Song Mei, Steve Mostovoy, Tong Mu, Gideon Myles, Alexander Neitz, Alex Nichol, Jakub Pachocki, Alex Paino, Dana Palmie, Ashley Pantuliano, Giambattista Parascandolo, Jongsoo Park, Leher Pathak, Carolina Paz, Ludovic Peran, Dmitry Pimenov, Michelle Pokrass, Elizabeth Proehl, Huida Qiu, Gaby Raila, Filippo Raso, Hongyu Ren, Kimmy Richardson, David Robinson, Bob Rotsted, Hadi Salman, Suvansh Sanjeev, Max Schwarzer, D. Sculley, Harshit Sikchi, Kendal Simon, Karan Singhal, Yang Song, Dane Stuckey, Zhiqing Sun, Philippe Tillet, Sam Toizer, Foivos Tsimpourlas, Nikhil Vyas, Eric Wallace, Xin Wang, Miles Wang, Olivia Watkins, Kevin Weil, Amy Wendling, Kevin Whinnery, Cedric Whitney, Hannah Wong, Lin Yang, Yu Yang, Michihiro Yasunaga, Kristen Ying, Wojciech Zaremba, Wenting Zhan, Cyril Zhang, Brian Zhang, Eddie Zhang, and Shengjia Zhao. gpt-oss-120b & gpt-oss-20b model card, 2025. URL https://arxiv.org/abs/2508.10925.

Long Phan, Alice Gatti, Ziwen Han, Nathaniel Li, Josephina Hu, Hugh Zhang, et al. Humanity's last exam. *arXiv preprint arXiv:2501.14249*, 2025.

Qwen. Qwen3.5: Accelerating productivity with native multimodal agents, February 2026. URL https://qwen.ai/blog?id=qwen3.5.

David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R. Bowman. GPQA: A graduate-level google-proof Q&A benchmark. *arXiv preprint arXiv:2311.12022*, 2023.

Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. Winogrande: An adversarial winograd schema challenge at scale. *Proceedings of the AAAI Conference on Artificial Intelligence*, 34(05):8732–8740, April 2020. ISSN 2159-5399. doi: 10.1609/aaai.v34i05.6399. URL http://dx.doi.org/10.1609/aaai.v34i05.6399.

Freda Shi, Mirac Suzgun, Markus Freitag, Xuezhi Wang, Suraj Srivats, Soroush Vosoughi, Hyung Won Chung, Yi Tay, Sebastian Ruder, Denny Zhou, Dipanjan Das, and Jason Wei. Language models are multilingual chain-of-thought reasoners, 2022. URL https://arxiv.org/abs/2210.03057.

Haoxiang Sun, Yingqian Chen, Xueyang Wen, Bingchen Hu, Tianyi Shi, Tianyi Wang, Junyi Wu, Wayne Xin Zhou, and Ji-Rong Wen. Challenging the boundaries of reasoning: An olympiad-level math benchmark for large language models. *arXiv preprint arXiv:2503.21380*, 2025.

Mirac Suzgun, Nathan Scales, Nathanael Schärli, Sebastian Gehrmann, Yi Tay, Hyung Won Chung, Aakanksha Chowdhery, Quoc V. Le, Ed H. Chi, Denny Zhou, and Jason Wei. Challenging big-bench tasks and whether chain-of-thought can solve them, 2022. URL https://arxiv.org/abs/2210.09261.

Jiaming Tang, Yilong Zhao, Kan Zhu, Guangxuan Xiao, Baris Kasikci, and Song Han. Quest: Query-aware sparsity for efficient long-context LLM inference. In *International Conference on Machine Learning (ICML)*, 2024.

Kimi Team, Yu Zhang, Zongyu Lin, Xingcheng Yao, Jiaxi Hu, Fanqing Meng, Chengyin Liu, Xin Men, Songlin Yang, Zhiyuan Li, Wentao Li, Enzhe Lu, Weizhou Liu, Yanru Chen, Weixin Xu, Longhui Yu, Yejie Wang, Yu Fan, Longguang Zhong, Enming Yuan, Dehao Zhang, Yizhi Zhang, T. Y. Liu, Haiming Wang, Shengjun Fang, Weiran He, Shaowei Liu, Yiwei Li, Jianlin Su, Jiezhong Qiu, Bo Pang, Junjie Yan, Zhejun Jiang, Weixiao Huang, Bohong Yin, Jiacheng You, Chu Wei, Zhengtao Wang, Chao Hong, Yutian Chen, Guanduo Chen, Yucheng Wang, Huabin Zheng, Feng Wang, Yibo Liu, Mengnan Dong, Zheng Zhang, Siyuan Pan, Wenhao Wu, Yuhao Wu, Longyu Guan, Jiawen Tao, Guohong Fu, Xinran Xu, Yuzhi Wang, Guokun Lai, Yuxin Wu, Xinyu Zhou, Zhilin Yang, and Yulun Du. Kimi linear: An expressive, efficient attention architecture, 2025a. URL https://arxiv.org/abs/2510.26692.

MiniCPM Team, Chaojun Xiao, Yuxuan Li, Xu Han, Yuzhuo Bai, Jie Cai, Haotian Chen, Wentong Chen, Xin Cong, Ganqu Cui, Ning Ding, Shengda Fan, Yewei Fang, Zixuan Fu, Wenyu Guan, Yitong Guan, Junshao Guo, Yufeng Han, Bingxiang He, Yuxiang Huang, Baoxi Ji, Cunliang Kong, Qiuzuo Li, Siyuan Li, Wenhao Li, Xin Li, Yanghao Li, Yishan Li, Zhen Li, Dan Liu, Biyuan Lin, Yankai Lin, Xiang Long, Quanyu Lu, Yaxi Lu, Peiyan Luo, Hongya Lyu, Litu Ou, Yinxu Pan, Lushi Pu, Zekai Qu, Qundong Shi, Zijun Song, Jiayuan Su, Zhou Su, Ao Sun, Xianghui Sun, Peijun Tang, Fangzheng Wang, Feng Wang, Shuo Wang, Yudong Wang, Zheng Wang, Yesai Wu, Zhenyu Xiao, Jie Xie, Zihao Xie, Xiaoyue Xu, Yukun Yan, Jiarui Yuan, Jinqian Zhang, Kaihuo Zhang, Lei Zhang, Linyue Zhang, Xueren Zhang, Yudi Zhang, Hengyu Zhao, Weilin Zhao, Weilun Zhao, Yuanqian Zhao, Zhi Zheng, Chuyue Zhou, Ge Zhou, Jie Zhou, Wei Zhou, Yanghao Zhou, Zihan Zhou, Zixuan Zhou, Zhiyuan

Liu, Guoyang Zeng, Chao Jia, Dahai Li, and Maosong Sun. Minicpm4: Ultra-efficient llms on end devices, 2025b. URL https://arxiv.org/abs/2506.07900.

Shengbang Tong, Ellis Brown, Penghao Wu, Sanghyun Woo, Manoj Middepogu, Sai Charitha Akula, Jihan Yang, Shusheng Yang, Adithya Iyer, Xichen Pan, Austin Wang, Rob Fergus, Yann LeCun, and Saining Xie. Cambrian-1: A fully open, vision-centric exploration of multimodal llms. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, editors, *Advances in Neural Information Processing Systems*, volume 37, pages 87310–87356. Curran Associates, Inc., 2024. doi: 10.52202/079017-2771. URL https://proceedings.neurips.cc/paper_files/paper/2024/file/9ee3a664ccfeabc0da16ac6f1f1cfe59-Paper-Conference.pdf.

Lei Wang, Yu Cheng, Yining Shi, Zhengju Tang, Zhiwen Mo, Wenhao Xie, Lingxiao Ma, Yuqing Xia, Jilong Xue, Fan Yang, and Zhi Yang. Tilelang: A composable tiled programming model for ai systems, 2025. URL https://arxiv.org/abs/2504.17577.

Yubo Wang, Xueguang Ma, Ge Zhang, Yuansheng Ni, Abhranil Chandra, Shiguang Guo, Weiming Ren, Aaran Arulraj, Xuan He, Ziyan Jiang, Tianle Li, Max Ku, Kai Wang, Alex Zhuang, Rongqi Fan, Xiang Yue, and Wenhu Chen. Mmlu-pro: A more robust and challenging multi-task language understanding benchmark. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, editors, *Advances in Neural Information Processing Systems*, volume 37, pages 95266–95290. Curran Associates, Inc., 2024a. doi: 10.52202/079017-3018. URL https://proceedings.neurips.cc/paper_files/paper/2024/file/ad236edc564f3e3156e1b2feafb99a24-Paper-Datasets_and_Benchmarks_Track.pdf.

Zirui Wang, Mengzhou Xia, Luxi He, Howard Chen, Yitao Liu, Richard Zhu, Kaiqu Liang, Xindi Wu, Haotian Liu, Sadhika Malladi, Alexis Chevalier, Sanjeev Arora, and Danqi Chen. CharXiv: Charting gaps in realistic chart understanding in multimodal LLMs. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2024b.

Haoning Wu, Dongxu Li, Bei Chen, and Junnan Li. Longvideobench: A benchmark for long-context interleaved video-language understanding. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, editors, *Advances in Neural Information Processing Systems*, volume 37, pages 28828–28857. Curran Associates, Inc., 2024. doi: 10.52202/079017-0907. URL https://proceedings.neurips.cc/paper_files/paper/2024/file/329ad516cf7a6ac306f29882e9c77558-Paper-Datasets_and_Benchmarks_Track.pdf.

Chaojun Xiao, Pengle Zhang, Xu Han, Guangxuan Xiao, Yankai Lin, Zhengyan Zhang, Zhiyuan Liu, Song Han, and Maosong Sun. InfLLM: Training-free long-context extrapolation for LLMs with an efficient context memory. *arXiv preprint arXiv:2402.04617*, 2024a.

Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. In *International Conference on Learning Representations (ICLR)*, 2024b.

Guangxuan Xiao, Junxian Guo, Kasra Mazaheri, and Song Han. Optimizing mixture of block attention. *arXiv preprint arXiv:2511.11571*, 2025.

Frank F. Xu, Yufan Song, Boxuan Li, Yuxuan Tang, Kritanjali Jain, Mengxue Bao, Zora Z. Wang, Xuhui Zhou, Zhitong Guo, Murong Cao, et al. TheAgentCompany: Benchmarking LLM agents on consequential real world tasks. *arXiv preprint arXiv:2412.14161*, 2024.

Ran Yan, Youhe Jiang, and Binhang Yuan. Flash sparse attention: More efficient natively trainable sparse attention. *arXiv preprint arXiv:2508.18224*, 2025.

Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated delta networks: Improving mamba2 with delta rule, 2025. URL https://arxiv.org/abs/2412.06464.

Howard Yen, Tianyu Gao, Minmin Hou, Ke Ding, Daniel Fleischer, Peter Izsak, Moshe Wasserblat, and Danqi Chen. HELMET: How to evaluate long-context models effectively and thoroughly. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=293V3bJbmE.

Jingyang Yuan, Huazuo Gao, Damai Dai, Junyu Luo, Liang Zhao, Zhengyan Zhang, Zhenda Xie, Y. X. Wei, Lean Wang, Zhiping Xiao, Yuqing Wang, Chong Ruan, Ming Zhang, Wenfeng Liang, and Wangding Zeng. Native sparse attention: Hardware-aligned and natively trainable sparse attention. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025. arXiv:2502.11089.

Xiang Yue, Yuansheng Ni, Kai Zhang, Tianyu Zheng, Ruoqi Liu, Ge Zhang, Samuel Stevens, Dongfu Jiang, Weiming Ren, Yuxuan Sun, Cong Wei, Botao Yu, Ruibin Yuan, Renliang Sun, Ming Yin, Boyuan Zheng, Zhenzhu Yang, Yibo Liu, Wenhao Huang, Huan Sun, Yu Su, and Wenhu Chen. MMMU: A massive multi-discipline multimodal understanding and reasoning benchmark for expert AGI. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.

Manzil Zaheer, Guru Guruganesh, Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontañón, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, and Amr Ahmed. Big bird: Transformers for longer sequences. In *Advances in Neural Information Processing Systems*, 2020.

Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. H2O: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems*, 2023.

Weilin Zhao, Zihan Zhou, Zhou Su, Chaojun Xiao, Yuxuan Li, Yanghao Li, Yudi Zhang, Weilun Zhao, Zhen Li, Yuxiang Huang, Ao Sun, Xu Han, and Zhiyuan Liu. Infllm-v2: Dense-sparse switchable attention for seamless short-to-long adaptation. *CoRR*, abs/2509.24663, 2025a. doi: 10.48550/ARXIV.2509.24663. URL https://doi.org/10.48550/arXiv.2509.24663.

Yilun Zhao, Lujing Xie, Haowei Zhang, Guo Gan, Yitao Long, Zhiyuan Hu, Tongyan Hu, Weiyuan Chen, Chuhan Li, Junyang Song, et al. MMVU: Measuring expert-level multi-discipline video understanding. *arXiv preprint arXiv:2501.12380*, 2025b.

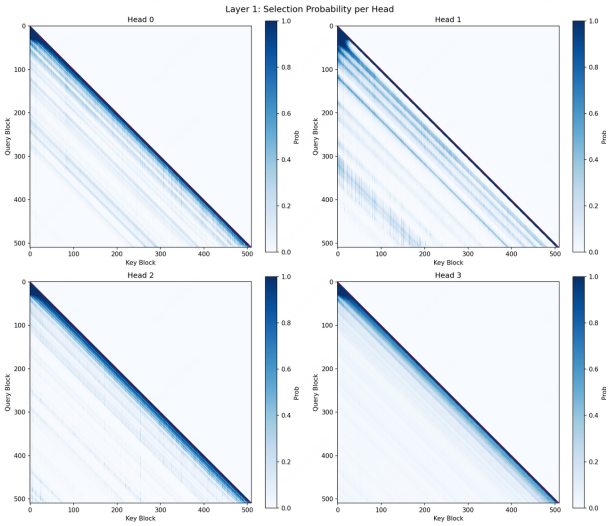
Zhipu AI. GLM-5.1: Open foundation models from Zhipu AI. https://github.com/THUDM/GLM-5, 2026.

Junjie Zhou, Yan Shu, Bo Zhao, Boya Wu, Zhengyang Liang, Shitao Xiao, Minghao Qin, Xi Yang, Yongping Xiong, Bo Zhang, Tiejun Huang, and Zheng Liu. Mlvu: Benchmarking multi-task long video understanding. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 13691–13701, June 2025.

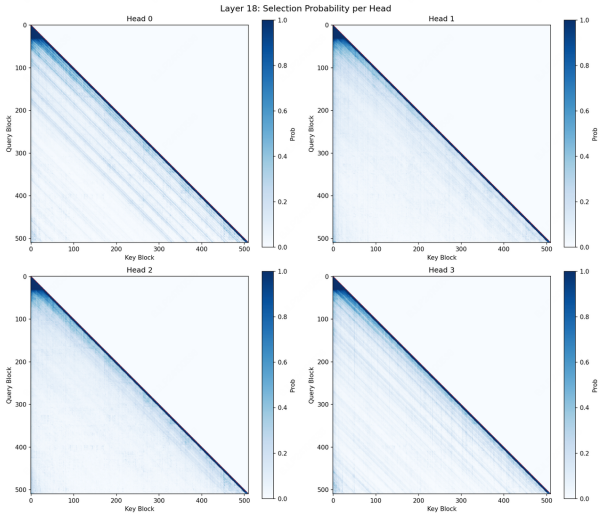
Terry Yue Zhuo, Vu Minh Chien, Jenny Chim, Han Hu, Wenhao Yu, Ratnadira Widyasari, Imam Nur Bani Yusuf, Haolan Zhan, Junda He, Indraneil Paul, Simon Brunner, Chen GONG, James Hoang, Armel Randy Zebaze, Xiaoheng Hong, Wen-Ding Li, Jean Kaddour, Ming Xu, Zhihan Zhang, Prateek Yadav, Naman Jain, Alex Gu, Zhoujun Cheng, Jiawei Liu, Qian Liu, Zijian Wang, David Lo, Binyuan Hui, Niklas Muennighoff, Daniel Fried, Xiaoning Du, Harm de Vries, and Leandro Von Werra. Bigcodebench: Benchmarking code generation with diverse function calls and complex instructions. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=YrycTjllL0.

A. Visualization

To better understand what the learned indexer selects, we visualize the per-head Index Branch selection probability over all query-block and key-block pairs in Figure 5. We show four heads from an early layer (Layer 1) and a later layer (Layer 18), corresponding to four different GQA groups. Across layers, the learned sparse pattern recovers the main structures expected from dense attention: all heads place high probability on the local diagonal, consistently select the sink column, and reserve the remaining budget for a small number of long-range relative positions. At the same time, the non-local selections are not identical across GQA groups. Different groups attend to different long-range stripes while sharing the common local and sink patterns, suggesting that the learned indexer captures group-specific sparse attention patterns rather than collapsing to a single global selection pattern.



(a) Layer 1, four GQA groups. Each group produces a different long-range selection pattern alongside the shared local diagonal and sink column.



(b) Layer 18, four GQA groups. Long-range selection sharpens into a few stripes per group; the four groups pick visibly different stripes.

Figure 5 | Per-head Index Branch selection probability across query-key pairs. Each panel shows four heads from one layer, corresponding to four different GQA groups. All groups consistently select the local diagonal and the sink column (leftmost), while different groups trace different long-range stripes, revealing group-specific sparse selection patterns.

We further examine the attention sink phenomenon in MSA models. Even without explicitly forcing the indexer to select the first key-value block, we observe that the learned Index Branch naturally assigns high selection probability to the initial block across all layers and heads. Figure 6 shows results for two representative layers (Layer 4 and Layer 24), each with eight sampled heads. Across both layers, every head directs a substantial fraction of its attention mass to the first token. This confirms that attention focal points naturally emerge and are universally present across different heads and layers, even in our sparse attention mechanism.

B. Preliminary Experiments

This section presents small-scale ablation studies on a pilot model. Our goal is to identify the training-design choices that are essential for stable optimization and strong downstream performance. These results serve as the empirical basis for the final recipe described in Section 3.

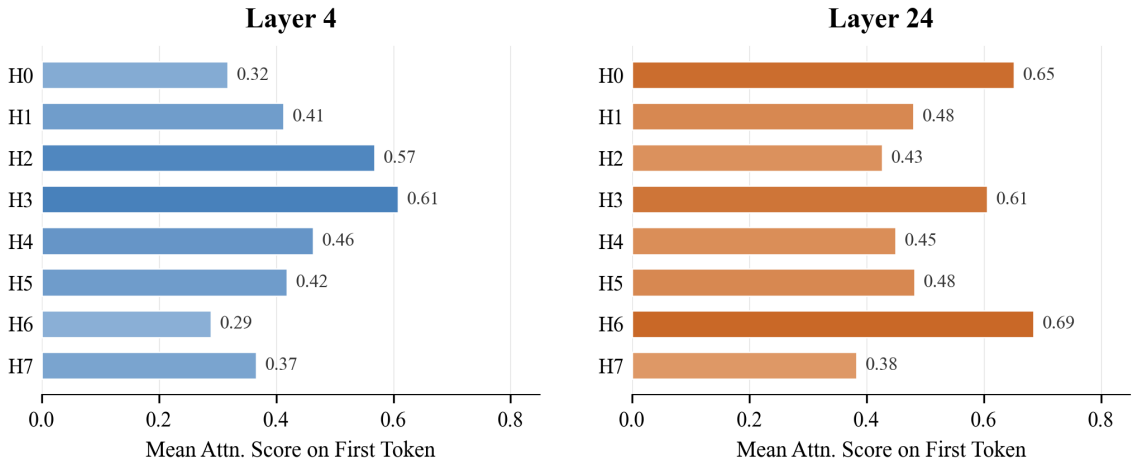


Figure 6 | Mean attention score on the first token for each attention head in Layer 4 and Layer 24. All heads allocate a significant fraction of attention to the first token, confirming a pervasive attention sink effect across heads and layers.

B.1. Setup

All ablations in this section use a 10B-parameter pilot Transformer with the same architecture family as the main paper MSA model but with 16 layers. The model uses a 200K-token vocabulary and hidden size $d_{\text{model}} = 2048$. Each attention module uses GQA with 32 query heads, 4 KV heads, head dimension 128, and RoPE dimension 64. The MoE contains 64 experts with top-4 expert routing and expert inner dimension 1536. The model has 10.53B total parameters and 1.47B active parameters per token. The optimizer, learning-rate schedule, and tokenizer match the full-scale configuration. Each run is trained on a subset of the same pretraining corpus used at full scale.

B.2. Gradient Sources for the Index Branch

A central challenge in training the Index Branch is that the top-k selection in Equation 7 is non-differentiable. Under the plain sparse-attention forward pass, the selected block indices are used only as a discrete routing decision. Consequently, the index projections $\boldsymbol{W}_q^{\text{idx}}$ and $\boldsymbol{W}_k^{\text{idx}}$ receive no useful gradient from the language-modelling objective, and the indexer cannot learn which blocks should be selected. There are several possible ways to introduce a training signal for the indexer. We investigate two mechanisms that preserve the sparse-attention structure while providing gradients to the Index Branch.

Index Branch output. The first mechanism lets the Index Branch contribute an additional attention output. Specifically, we attach a value projection to the Index Branch and compute $\boldsymbol{O}^{\text{idx}} = \text{Attn}(\boldsymbol{Q}^{\text{idx}}, \boldsymbol{K}^{\text{idx}}, \boldsymbol{V}^{\text{idx}}) \in \mathbb{R}^{N \times H_q \times d_h}$. This output is added to the layer output through a separate output projection, $\boldsymbol{O}' = \boldsymbol{W}_o \boldsymbol{O} + \boldsymbol{W}_o^{\text{idx}} \boldsymbol{O}^{\text{idx}}$. This design trains the Index Branch through its contribution to next-token prediction.

KL loss. The second mechanism directly supervises the Index Branch by matching its selection distribution to the Main Branch on the selected support. We use the auxiliary loss $\mathcal{L}_{\text{KL}}$ defined in Equation 10. This loss acts on $\boldsymbol{W}_q^{\text{idx}}$ and $\boldsymbol{W}_k^{\text{idx}}$, and provides an explicit training signal for the index selection.

To separate the effects of these two gradient sources, we train the model from scratch in three configurations, using sparse attention from the first step:

- **LM Loss only**: the Index Branch output is added to the layer output, and the model is trained only with the language-modelling loss,

$$\boldsymbol{O}' = \boldsymbol{W}_o\boldsymbol{O} + \boldsymbol{W}_o^{\text{idx}}\boldsymbol{O}_{\text{idx}}, \qquad \mathcal{L} = \mathcal{L}_{\text{LM}}. \tag{17}$$

- **KL Loss only**: the Index Branch output is discarded, and the indexer is trained only through the auxiliary KL loss,

$$\boldsymbol{O}' = \boldsymbol{W}_o\boldsymbol{O}, \qquad \mathcal{L} = \mathcal{L}_{\text{LM}} + \lambda \sum_{\text{layers}} \mathcal{L}_{\text{KL}}. \tag{18}$$

- **LM Loss + KL Loss**: both mechanisms are enabled,

$$\boldsymbol{O}' = \boldsymbol{W}_o\boldsymbol{O} + \boldsymbol{W}_o^{\text{idx}}\boldsymbol{O}_{\text{idx}}, \qquad \mathcal{L} = \mathcal{L}_{\text{LM}} + \lambda \sum_{\text{layers}} \mathcal{L}_{\text{KL}}. \tag{19}$$

Figure 7 reports the per-benchmark delta of each configuration against the Full-Attention GQA baseline trained on the same data. The two single-signal configurations show complementary weaknesses. **LM Loss only** preserves short-context ability but performs poorly on long-context retrieval: without an objective on the top-k selection itself, the indexer receives little direct pressure to select relevant blocks. **KL Loss only** improves retrieval but reduces short-context ability: removing $\boldsymbol{O}_{\text{idx}}$ from the layer output reduces the attention capacity available to the language model. **LM Loss + KL Loss** gives the best balance across the two axes and is the configuration we use for the remaining ablations in this section.

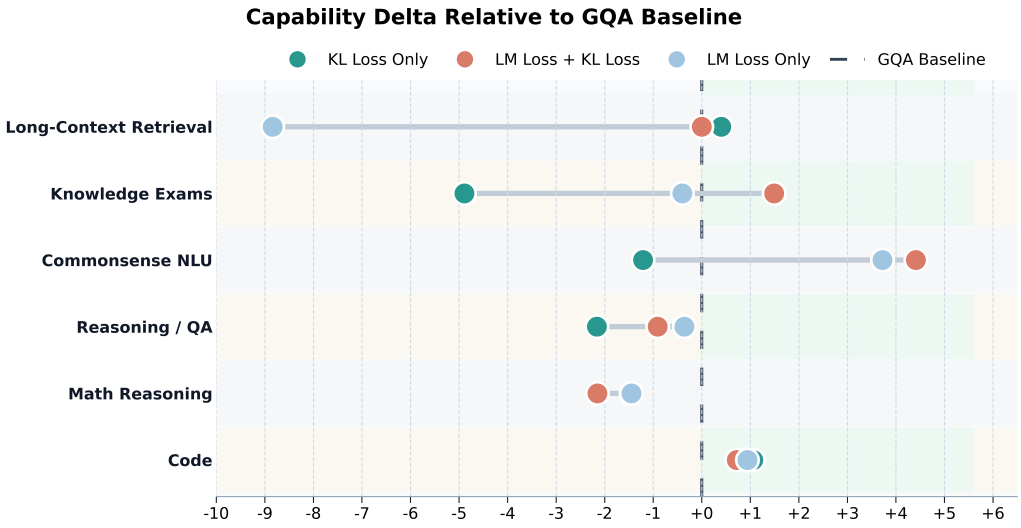


Figure 7 | Evaluation-score deltas relative to the Full-Attention baseline for three indexer training signals in the pilot setting. Positive values indicate improvements over the baseline, and negative values indicate degradations.

Based on these results, we use the **LM Loss + KL Loss** configuration for the remaining ablations in this section. We later show in Section C.3 that, once the indexer warmup introduced in Section B.4 is used in the full-scale setting, the Index Branch output is no longer necessary. The final recipe, therefore, keeps the KL supervision but removes the Index Branch value head and its additive output path.

B.3. Confining the KL Gradient to the Index Branch

The auxiliary KL loss is intended to train the Index Branch to match the Main Branch selection distribution. Under the default autograd graph, the KL gradient flows through the Index Branch query and key projections back into the hidden state, and then into the backbone through the residual stream. In this case, the KL loss becomes an additional objective on the backbone, rather than a local supervision signal for the indexer.

We observe two failure modes from this gradient routing. With larger KL coefficients, occasional KL-gradient spikes propagate into the backbone, causing gradient-norm spikes and LM-loss divergence within a few hundred steps (Figure 8). Even at stable coefficients, standard short-context benchmarks gradually regress during training (Figure 9). We attribute this regression to a self-distillation effect: the backbone can lower the KL loss by simplifying the Main Branch attention distribution, rather than by improving the Index Branch.

We address both failure modes by stopping the KL gradient at the Index Branch input (Section 3.2). Thus, each layer's KL loss becomes a local supervision signal for its own indexer. With this detach, the LM loss and gradient norm remain stable under the same KL coefficients that cause divergence without detach (Figure 8), and the short-context regression is removed (Figure 9). We use this detach in all subsequent runs.

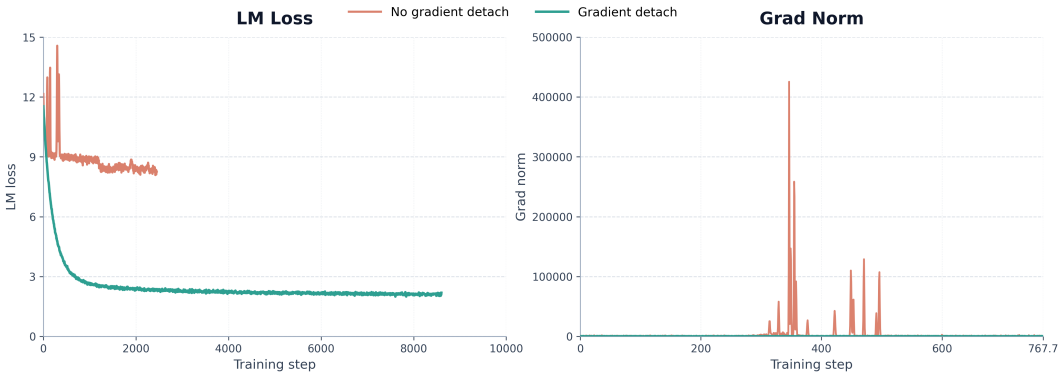


Figure 8 | Training LM loss and gradient norm with and without detaching the KL gradient from the backbone. Detaching confines the auxiliary loss to the Index Branch and avoids the gradient spikes observed without detach.

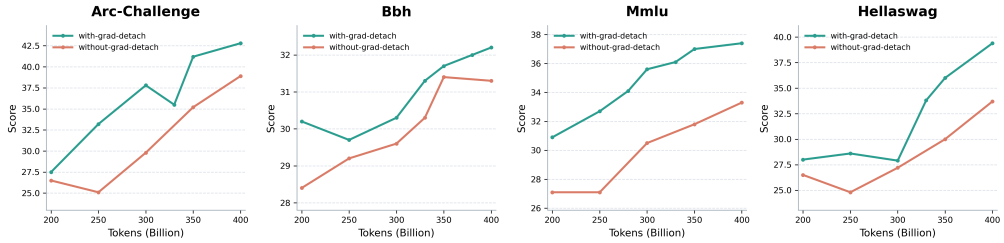


Figure 9 | General benchmark scores with and without detaching the KL gradient from the backbone. Detaching the auxiliary loss reduces the general ability degeneration observed when the KL gradient updates the backbone.

B.4. Indexer Warmup

We observe that the Main Branch attention distribution changes rapidly during the earliest stage of training. As shown in Figure 10, the attention entropy quickly drops from an initially smooth distribution to a much sharper one, before entering a slower phase of representation learning. This makes sparse selection fragile at initialization. If top-k selection is enabled from step zero, the Index Branch must track a rapidly moving target while its own selections are still nearly random. Poor early selections then route the Main Branch to uninformative tokens, which weakens both backbone learning and the KL supervision received by the indexer.

We address this issue with a short indexer warmup. During warmup, the Main Branch uses full attention, while the Index Branch is trained by the KL loss against the full-sequence Main Branch distribution. This allows the backbone to pass through the early sharpening phase without sparse routing errors, and gives the indexer a meaningful initialization before it controls token selection. After T_{warm} steps, we enable top-k sparse selection and continue training with the KL loss restricted to the selected support.

Figure 11 compares pretraining runs with and without this warmup. The warmed-up run achieves better short-context performance and stronger long-context retrieval. These results indicate that a short full-attention warmup provides a better initialization for sparse training. We therefore also adopt this warmup when converting Full-Attention checkpoints to sparse attention through continued pretraining.

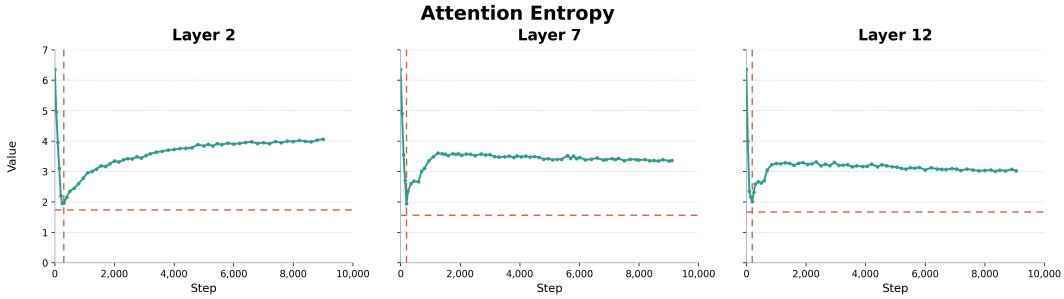


Figure 10 | Per-layer entropy of the Main Branch attention distribution during early sparse training. Entropy drops rapidly in the first few hundred steps before partially recovering and stabilizing, motivating a brief full-attention warmup for the indexer.

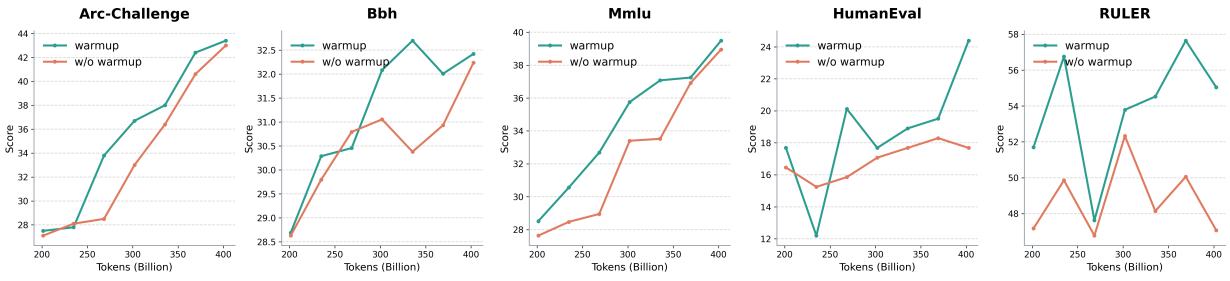


Figure 11 | Evaluation results of MSA with and without index warmup. Within the reported training range, index warmup improved scores on general tasks and long-context retrieval.

B.5. Learnable Attention Sink

The visualization in Figure 6 shows that the first token often acts as an attention sink: many heads assign a non-trivial amount of attention mass to the sequence prefix, even when the sparse selector is not explicitly forced to include it. This raises the question of whether this sink behavior should be represented by an explicit learnable mechanism, rather than being absorbed by the first real token in the sequence. We therefore tested a GPT-OSS-style learnable attention sink. Concretely, each attention head is given an additional learnable sink logit, which competes with normal key positions in the attention softmax.

Figure 12 visualizes the resulting attention patterns. The learnable sink absorbs substantial attention mass in some heads, but it does not completely remove the original first-token sink. In several heads, especially those where the learned sink receives little mass, the first token still receives substantial attention and continues to behave as an implicit sink.

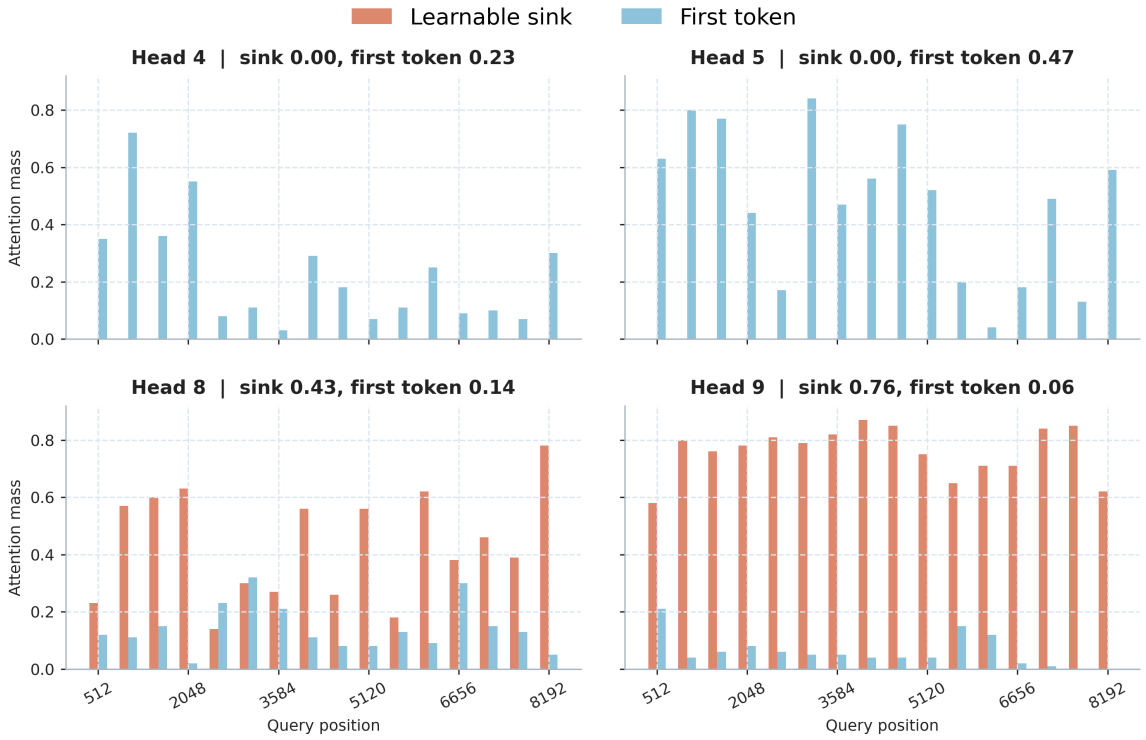


Figure 12 | Attention received by the learnable sink and the first token after introducing a GPT-OSS-style sink parameter. In some heads, the learnable sink absorbs most of the sink-like attention; in others, the first token remains the dominant sink, indicating that the explicit sink does not fully eliminate first-token sink behavior.

We also compare downstream perplexity with and without the learnable sink in Figure 13. The learnable-sink variant does not yield a clear or consistent improvement over the default design. Given its additional parameters, implementation complexity, and the fact that it does not fully suppress first-token sink behavior, we do not include the learnable attention sink in the final recipe.

B.6. Dynamic Sparse Selection vs. Sliding Window

To assess the value of dynamic selection, we compare MSA with a FLOP-matched sliding-window baseline. This baseline removes the Index Branch and uses a fixed sparse pattern: each query attends

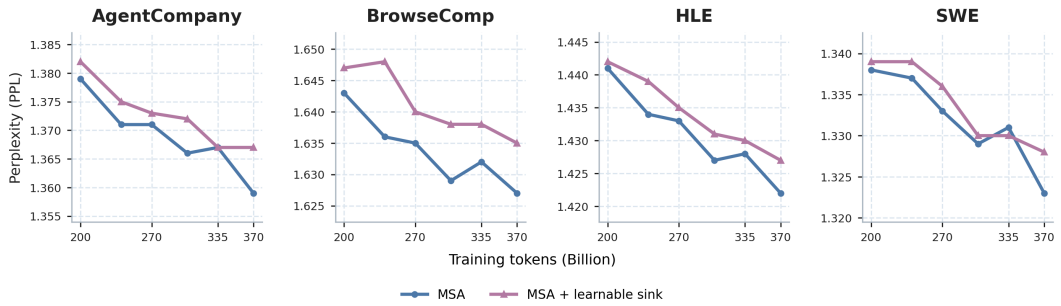


Figure 13 | Perplexity comparison with and without the learnable attention sink on downstream agent-oriented evaluations. Lower perplexity is better. Adding the learnable sink does not provide a consistent advantage over the default MSA design.

to the first key block and to a local window with the same token budget ending at the query. Therefore, the two methods have the same selection budget and differ only in whether the selected tokens are fixed by position or chosen dynamically.

Figure 14 reports perplexity on downstream agent tasks. Under the same sparse selection budget, the sliding-window model has higher perplexity than MSA across the training trajectory. Although both models benefit from additional training tokens, the fixed local-window pattern does not match the perplexity of dynamic sparse selection. This suggests that, for these agent tasks, a position-fixed sparse pattern is less suitable than content-dependent token selection.

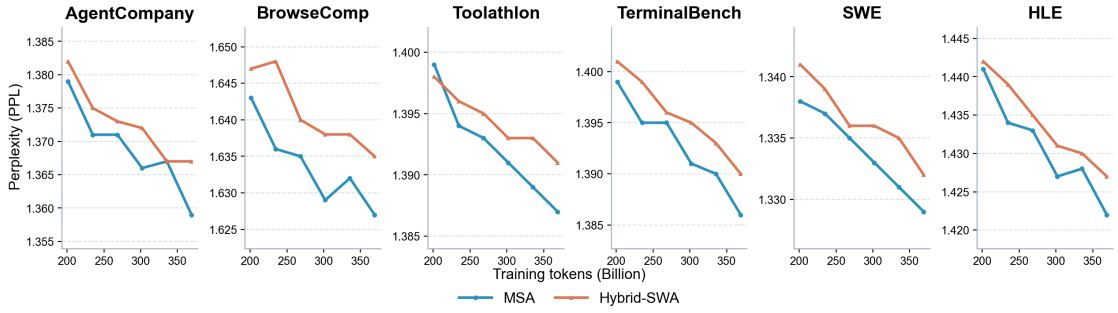


Figure 14 | Perplexity comparison between MSA and a FLOP-matched sliding-window baseline on downstream agent-oriented evaluations. Lower Perplexity indicates better modeling performance under the same sparse selection budget.

C. Additional Ablation Study

C.1. Block Size

The sparse attention calculation in MSA's Main Branch processes key-value pairs in units of consecutive B_k token blocks, which affects both model performance and efficiency. Larger blocks can improve kernel efficiency but may reduce retrieval quality because of coarser selection granularity. By adjusting B_k while keeping the total number of selected tokens constant, we investigate this trade-off. Compared to the main experiment, these runs use fewer training iterations and a subset of the evaluation suite.

As shown in Table 4, varying the block size has a limited impact on model quality in this setting. The PPL results are nearly unchanged across different B_k values, and the RULER scores show no clear

degradation when increasing the block size from 32 to 64 or 128. This suggests that MSA can use larger key-value blocks to improve kernel efficiency with limited quality loss in these ablations.

Table 4 | Perplexity and long-context retrieval scores for different key-value block sizes. Lower is better for perplexity, and higher is better for RULER scores.

Benchmark	Block 32	Block 64	Block 128
PPL ↓			
TAU2	1.176	1.176	1.176
AgentCompany	1.266	1.276	1.266
HLE	1.299	1.299	1.300
SWE	1.233	1.233	1.233
Long-context retrieval			
RULER-8K	72.5	72.8	73.8
RULER-32K	66.1	65.3	64.6

C.2. Forced Sink & Local Selection

In early sparse-training experiments, we explicitly forced the selector to include two types of blocks: the first block in the sequence and a fixed local window around the query position. The first block corresponds to the common attention-sink pattern, while the local window preserves nearby context that is important for short-range modeling and provides dense supervision for the indexer. This design was mainly introduced as a stabilization mechanism: before the indexer becomes reliable, forcing these blocks reduces the chance that the sparse branch misses basic context during early training.

We later found that these priors do not need to be hard-coded. When the forced selection of the first block and the fixed local window are removed, the trained model still exhibits both structures: attention concentrates on the sequence prefix when useful, and nearby tokens remain frequently selected. As shown in Table 5, removing forced sink and fixed local selection has little effect on standard model quality: reasoning, code, and PPL metrics remain nearly unchanged. Long-context retrieval is also comparable. These results indicate that the sparse model can learn sink and local-selection patterns without hard-coded selection rules. Therefore, the final recipe does not force the first block or a large local window, and only forces the special incomplete self block.

C.3. Index Branch Value Head

Our preliminary experiments (Section B.2) show that providing an additional attention output through the Index Branch helps the model begin sparse training from step zero. However, this index value head introduces additional computation and complexity. Since the indexer warmup in Section B.4 already improves the initialization for sparse training, we further ablate whether the value head is still needed.

We compare the original with-value design against a no-value variant that trains the indexer only with the KL alignment signal. As shown in Table 6, removing the index value head does not lead to a systematic degradation across the evaluation suite. The no-value variant is slightly better on some general reasoning benchmarks, while the with-value variant retains small advantages on some math and code tasks. On multimodal benchmarks and long-context retrieval, the differences are also mixed.

Overall, the results indicate that the index value head is not critical once the Index Branch warmup is used. Its effect on downstream quality is small and benchmark-dependent, with neither variant

Table 5 | Ablation of forced sink and local-window selection. Higher is better unless marked ↓.

Benchmark	No Forced	Forced
General knowledge & reasoning		
MMLU	60.5	60.5
MMLU-Pro	32.5	33.4
BBH	58.2	58.2
ARC Challenge	78.1	77.9
Math		
GSM8K	66.0	66.9
MGSM	35.8	36.3
Code		
EvalPlus	54.0	53.6
BigCodeBench	35.6	35.7
MultiPL-E MBPP P@10	80.1	79.5
Image		
ChartQA	73.5	73.7
MMMU	43.6	42.9
Video		
VideoMMMU	32.1	32.0
PPL ↓		
TAU2	1.175	1.175
AgentCompany	1.268	1.266
HLE	1.301	1.300
SWE	1.235	1.233
Long-context retrieval		
RULER-8K	71.6	71.7
RULER-32K	61.5	65.8

Table 6 | Continued pre-training ablation of the Index Branch value head.

Benchmark	With-value	No-value
General knowledge & reasoning		
MMLU	66.4	67.3
MMLU-Pro	39.0	39.1
BBH	65.3	65.9
ARC Challenge	82.2	82.4
Math		
GSM8K	77.6	76.4
MathVista	45.2	43.6
MGSM	48.4	47.6
Code		
HumanEval	60.4	59.1
EvalPlus	57.7	58.7
BigCodeBench	46.0	44.0
Image		
AI2D	69.3	70.4
ChartQA	75.3	74.9
MMMU	44.9	43.4
OCRBench v2	53.2	53.9
Video		
MLVU	42.4	43.9
MMVU	44.9	43.7
PerceptionTest	45.0	47.3
Long-context retrieval		
RULER-8K	84.1	83.0
RULER-32K	79.7	80.4

consistently dominating the other. This suggests that the main role of $\boldsymbol{O}_{\text{idx}}$ in the earlier recipe was to provide an additional early training signal, rather than to supply essential capacity at convergence. The final design, therefore, drops the index value head on efficiency grounds. At inference time, the top-k indexer only needs the block-wise maximum of $\boldsymbol{Q}_{\text{idx}}\boldsymbol{K}_{\text{idx}}^{\top}$, avoiding the value aggregation path and exponential calculations entirely.